



THALLYS HENRIQUE MARTINS

**DESENVOLVIMENTO E EXPERIMENTAÇÃO DE UM
PROTÓTIPO DE FEEDBACK AUTOMATIZADO PARA
CORREÇÃO DE EXERCÍCIOS DE PROGRAMAÇÃO
ORIENTADA A OBJETOS USANDO GRANDES MODELOS DE
LINGUAGEM**

LAVRAS – MG

2026

THALLYS HENRIQUE MARTINS

**DESENVOLVIMENTO E EXPERIMENTAÇÃO DE UM PROTÓTIPO DE FEEDBACK
AUTOMATIZADO PARA CORREÇÃO DE EXERCÍCIOS DE PROGRAMAÇÃO
ORIENTADA A OBJETOS USANDO GRANDES MODELOS DE LINGUAGEM**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

**Ficha Catalográfica elaborada pelo Sistema de Geração
de Ficha Catalográfica da Biblioteca Universitária da UFLA, com
dados informados pelo(a) próprio(a) autor(a).**

Martins, Thallys.

Desenvolvimento e Experimentação de um Protótipo de Feedback Automatizado
para Correção de Exercícios de
Programação Orientada a Objetos usando Grandes Modelos de Linguagem / Thallys
Martins. - 2026.

59 p. : il.

Orientador: Julio César Alves

Trabalho de Conclusão de Curso (Graduação) - Universidade Federal de Lavras,
2026.

Bibliografia.

1. Programação orientada a objetos. 2. Grandes modelos de linguagem. 3.
Feedback pedagógico. 4. Avaliação automática de código. 5. Inteligência artificial na
educação. I. Alves, Julio César. II. Universidade Federal de Lavras. III. Título.

THALLYS HENRIQUE MARTINS

**DESENVOLVIMENTO E EXPERIMENTAÇÃO DE UM PROTÓTIPO DE FEEDBACK
AUTOMATIZADO PARA CORREÇÃO DE EXERCÍCIOS DE PROGRAMAÇÃO
ORIENTADA A OBJETOS USANDO GRANDES MODELOS DE LINGUAGEM**

**DEVELOPMENT AND EXPERIMENTATION OF AN AUTOMATED FEEDBACK
PROTOTYPE FOR THE CORRECTION OF OBJECT-ORIENTED PROGRAMMING
EXERCISES USING LARGE LANGUAGE MODELS**

Trabalho de Conclusão de Curso apresentado
à Universidade Federal de Lavras, como
parte das exigências do Curso de Ciência da
Computação, para a obtenção do título de
Bacharel

APROVADA em 19 de junho de 2026.

Prof. Dr. Jesimar da Silva Arantes

DAC-UFLA

Prof. Dr. Luiz Henrique de Campos Merschmann

DAC-UFLA

Documento assinado digitalmente
 JULIO CESAR ALVES
Data: 26/06/2026 09:02:41-0300
Verifique em <https://validar.iti.gov.br>

Prof. Dr. Julio César Alves
Orientador

**LAVRAS – MG
2026**

AGRADECIMENTOS

A concretização deste Trabalho de Conclusão de Curso não seria possível sem o apoio, a paciência e o incentivo de pessoas e instituições que me acompanharam ao longo desta caminhada acadêmica.

À minha família, pelo apoio incondicional, compreensão e incentivo ao longo de toda a minha jornada.

Ao meu orientador, Prof. Dr. Julio César Alves, pela paciência, confiança e dedicação na condução deste trabalho.

Agradeço a cada professor, mestre e colega que cruzou o meu caminho, compartilhando sua sabedoria e proporcionando os ensinamentos fundamentais para a minha evolução pessoal e profissional.

Aos meus amigos, que estiveram ao meu lado oferecendo ajuda, motivação e bons conselhos sempre que necessário.

Por fim, à Universidade Federal de Lavras, pela oportunidade de formação e por todo o suporte institucional fornecido.

RESUMO

A avaliação de exercícios práticos em disciplinas de Programação Orientada a Objetos (POO) é fundamental para o aprendizado, mas exige um alto esforço do docente, especialmente em relação à consistência e clareza do *feedback*. Ferramentas de correção automática são valiosas, mas muitas vezes limitadas na qualidade do retorno fornecido ao estudante. Com o avanço dos Grandes Modelos de Linguagem (*Large Language Models* – LLMs), torna-se viável a criação de um sistema de correção que ofereça *feedback* personalizado e sofisticado, superando as limitações dos sistemas tradicionais. Este trabalho apresenta uma abordagem inicial de utilização de LLMs para correção e fornecimento de *feedback* em exercícios de POO, baseada no uso da biblioteca LangGraph, que fornece flexibilidade na escolha do modelo de IA (LLM) a ser utilizado. Os experimentos realizados fornecem uma base para o avanço da pesquisa na área, e apresentam *insights* valiosos para esse tipo de ferramenta.

Palavras-chave: programação orientada a objetos; grandes modelos de linguagem; feedback pedagógico; avaliação de código.

ABSTRACT

The evaluation of practical exercises in Object-Oriented Programming (OOP) disciplines is fundamental for learning, but it demands high faculty effort, especially regarding the consistency and clarity of feedback. Automated grading tools are valuable, but often limited in the quality of the return provided to the student. With the advancement of Large Language Models (LLMs), it becomes feasible to create a grading system that offers personalized and sophisticated feedback, overcoming the limitations of traditional systems. This work presents an initial approach to utilizing LLMs for grading and providing feedback in OOP exercises, based on the use of the LangGraph library, which provides flexibility in choosing the AI model (LLM) to be used. The experiments conducted provide a foundation for the advancement of research in the field and offer insights that can be valuable for this type of tool.

Keywords: object-oriented programming; large language models; pedagogical feedback; code evaluation.

LISTA DE FIGURAS

Figura 4.1 – Fluxo geral de funcionamento do sistema.	23
Figura 4.2 – Diagrama de blocos da arquitetura modular e fluxo centralizado de componentes.	24
Figura 4.3 – Fluxo de estados implementado com LangGraph.	24
Figura 4.4 – Painel de controle da interface gráfica para parametrização de submissões. .	30
Figura 4.5 – Detalhe da interface gráfica para seleção e priorização de conceitos de POO.	31
Figura 4.6 – Interface gráfica do relatório pedagógico em HTML (Parte Superior). . . .	32
Figura 4.7 – Interface gráfica do relatório pedagógico em HTML (Parte Inferior). . . .	33

SUMÁRIO

1	INTRODUÇÃO	9
1.1	Objetivos	10
1.2	Estrutura do Texto	10
2	REFERENCIAL TEÓRICO	12
2.1	Ensino de Programação Orientada a Objetos	12
2.2	Correção Automática de Código	13
2.3	Grandes Modelos de Linguagem (LLMs)	14
2.4	IA como Tutor no Processo de Aprendizagem	15
2.5	Tutoria Inteligente e Estratégias de <i>Scaffolding</i>	16
3	METODOLOGIA	17
3.1	Identificação da Pesquisa	17
3.2	Contexto da Investigação	17
3.3	O Problema da “Correção Estrutural Binária”	17
3.4	Procedimentos Metodológicos	18
3.5	Ecossistema Tecnológico e Configuração do Ambiente	19
4	DESENVOLVIMENTO DO SISTEMA	21
4.1	Arquitetura do Sistema	21
4.1.1	Estrutura de Diretórios e Módulos	22
4.1.2	Fluxo Geral de Funcionamento do Sistema	23
4.1.3	Fluxo de Componentes	24
4.1.4	Fluxo de Processamento no LangGraph	24
4.2	Tecnologias e Ferramentas	25
4.3	Funcionalidades Principais	26
4.4	Processo de Implementação e Desafios	27
4.5	Interface Gráfica e Relatórios do Sistema	29
4.5.1	Interface Gráfica do Usuário	29
4.5.2	Estrutura do Relatório Pedagógico Gerado	31
4.6	Procedimentos de Teste e Validação	34
5	RESULTADOS E DISCUSSÃO	35
5.1	Contextualização dos Cenários de Teste	35
5.2	Primeira Estratégia de Prompt: Arquitetura Inicial e Premissas	36

5.3	Avaliação da Primeira Estratégia: Estudos de Caso e Diretrizes	37
5.4	Análise Qualitativa e Limitações Identificadas	38
5.4.1	Consistência Pedagógica e Organização Estrutural do Código	38
5.4.2	Limitações Técnicas e Rigor Excessivo	38
5.5	Evolução do Feedback: Segunda Estratégia de Prompt	39
5.5.1	Avaliação Ampliada em Submissões Reais	42
5.6	Limitações Técnicas e Operacionais do Protótipo	45
6	CONCLUSÃO	48
	REFERÊNCIAS	50
	ANEXOS	52
	APÊNDICES	54

1 INTRODUÇÃO

A crescente demanda por profissionais qualificados na área de tecnologia tem impulsionado a busca por métodos de ensino de programação mais escaláveis e eficientes. No contexto do ensino superior, as disciplinas de Programação Orientada a Objetos (POO) representam um marco na formação do estudante, exigindo a transição do pensamento puramente algorítmico para uma visão baseada em *design*, abstração e arquitetura de sistemas. No entanto, a avaliação qualitativa de exercícios nesse paradigma apresenta desafios significativos.

O processo de fornecer *feedback* detalhado e personalizado é uma tarefa que consome um volume considerável de tempo do docente. De acordo com Pereira e Mello (2025), a falta de um retorno pedagógico imediato e consistente pode prejudicar o ciclo de aprendizagem do discente, especialmente quando os erros cometidos não são apenas sintáticos, mas conceituais, envolvendo princípios fundamentais do paradigma, como encapsulamento e coesão (Cipriano; Alves, 2023).

Ferramentas de correção automática tradicionais, embora úteis para validar se um código compila ou passa em testes unitários, frequentemente falham em oferecer uma análise qualitativa profunda. Estas ferramentas costumam atuar como validadores estruturais, focando no “que” o código faz e negligenciando o “como” ele foi projetado. Com a recente popularização dos Grandes Modelos de Linguagem (LLMs), abre-se uma nova fronteira para a automação pedagógica, permitindo a geração de *feedbacks* que se assemelham à análise realizada por um tutor humano.

Entretanto, a integração direta de LLMs em sistemas de ensino esbarra em obstáculos técnicos, como a tendência desses modelos a gerar informações imprecisas (alucinações) e a necessidade crítica de supervisão humana para viabilizar a sustentabilidade pedagógica (Pereira; Mello, 2025). Além disso, a rápida evolução tecnológica torna arriscado o desenvolvimento de sistemas dependentes de um único provedor de inteligência artificial.

Este trabalho propõe o desenvolvimento e a experimentação de um protótipo de *feedback* automatizado construído sobre uma arquitetura desacoplada utilizando o *framework* Lang-Graph¹ para coordenação do fluxo de análise e geração de *feedback*. Embora a arquitetura tenha sido projetada para facilitar a integração de diferentes modelos generativos por meio de uma ca-

¹ Disponível em: <<https://www.langchain.com/langgraph>>. Acesso em: 9 jun. 2026.

mada de abstração, a validação experimental desta pesquisa utilizou exclusivamente o modelo Gemini 2.5 Flash via SDK oficial.

O protótipo desenvolvido neste trabalho foi disponibilizado como software livre sob a licença *MIT*. Isso permite a reutilização, modificação e continuidade do projeto por parte da comunidade acadêmica.²

1.1 Objetivos

O objetivo deste trabalho é desenvolver e avaliar um protótipo de sistema de correção de código para exercícios de POO utilizando a biblioteca LangGraph, com foco na flexibilidade técnica e na qualidade do *feedback* gerado por IA. Para isso, os seguintes objetivos específicos foram estabelecidos:

- Desenvolver a arquitetura e o fluxo principal de correção e análise utilizando uma abordagem baseada em grafos, proporcionando flexibilidade técnica para alternar e comparar diferentes modelos de linguagem;
- Implementar rotinas automatizadas para enriquecer a avaliação, transformando análises quantitativas em *feedback* qualitativo e claro para os estudantes;
- Utilizar inteligência artificial para avaliar a corretude semântica e estrutural do código-fonte, identificando falhas conceituais no paradigma de orientação a objetos;
- Validar a eficácia, precisão e consistência das correções e dos *feedbacks* gerados em um conjunto de exercícios práticos de Programação Orientada a Objetos;
- Analisar e documentar as limitações técnicas e comportamentais identificadas nos testes para avaliar a viabilidade prática do sistema.

1.2 Estrutura do Texto

Este trabalho está organizado em seis capítulos. O Capítulo 2 aborda o referencial teórico, discutindo os desafios no ensino de POO, as limitações dos sistemas tradicionais de

² O código-fonte completo do sistema, histórico de desenvolvimento e instruções de execução estão disponíveis em: <https://github.com/Kyutz/langgraph-correcao-poc>

avaliação automática e os fundamentos das LLMs e da arquitetura Transformer. O Capítulo 3 apresenta a metodologia adotada, descrevendo o contexto acadêmico em que os experimentos foram conduzidos, o problema da correção estrutural binária e os procedimentos metodológicos executados ao longo da pesquisa. O Capítulo 4 descreve o desenvolvimento do sistema, detalhando a arquitetura modularizada, as tecnologias utilizadas, a interface gráfica construída e as soluções implementadas para contornar desafios práticos do código. O Capítulo 5 apresenta os resultados e discussões, trazendo estudos de caso práticos para analisar qualitativamente a evolução do comportamento do sistema diante das estratégias de *prompt* testadas, além de documentar as limitações técnicas e operacionais identificadas nos experimentos. Por fim, o Capítulo 6 apresenta as conclusões obtidas e as propostas para trabalhos futuros.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os conceitos e trabalhos que fundamentam a pesquisa desenvolvida. Inicialmente são discutidos os desafios relacionados ao ensino de Programação Orientada a Objetos e às limitações dos métodos tradicionais de correção automática. Em seguida, são apresentados os fundamentos dos Grandes Modelos de Linguagem e seu potencial de aplicação em contextos educacionais. Por fim, são analisados trabalhos relacionados que empregam inteligência artificial como ferramenta de apoio ao ensino e à geração de *feedback*.

2.1 Ensino de Programação Orientada a Objetos

O ensino de Programação Orientada a Objetos configura-se como um dos maiores desafios nos currículos de Ciência da Computação. Diferente do paradigma estruturado, que foca em sequências de instruções e lógica procedimental, a POO exige que o discente desenvolva uma capacidade de abstração para modelar entidades do mundo real em componentes de software interconectados.

De acordo com Booch, Maksimchuk e Engle (2007), o modelo de objetos fundamenta-se em quatro elementos essenciais: abstração, encapsulamento, modularidade e hierarquia. O autor defende que o domínio desses conceitos é o que permite gerenciar a complexidade inerente ao desenvolvimento de software moderno, enfatizando que a orientação a objetos transcende a sintaxe das linguagens de programação e deve ser compreendida como um paradigma de *design* e organização de sistemas.

A transição para o paradigma orientado a objetos impõe um desafio cognitivo significativo, visto que estudantes iniciantes frequentemente enfrentam barreiras para consolidar o “pensar em objetos”. Segundo Eckerdal e Berglund (2005), o aprendizado da POO exige que o aluno ultrapasse a compreensão meramente sintática da linguagem para atingir um nível de abstração que permita a modelagem de problemas complexos.

Complementarmente, Armstrong (2006) argumenta que essa transição é dificultada quando o ensino foca excessivamente na implementação em detrimento dos conceitos fundamentais — os “*quarks*” do desenvolvimento. Para a autora, a plena compreensão do paradigma exige o domínio de conceitos de estrutura (como objetos, classes, herança e encapsulamento) e de comportamento (como polimorfismo e passagem de mensagens). Sem essa base conceitual, o

aluno tende a replicar vícios da lógica procedimental, utilizando classes apenas como depósitos de dados e manipulando atributos de forma externa. Ao falhar em delegar responsabilidades aos objetos através de métodos e troca de mensagens, o estudante produz sistemas que, embora funcionalmente corretos, carecem das vantagens de manutenibilidade e reuso típicas de arquiteturas orientadas a objetos.

Diferente de disciplinas introdutórias, que frequentemente recorrem a ferramentas de correção automática baseadas em testes de saída — a exemplo do Dredd (Schneider, 2021), desenvolvido e amplamente utilizado no contexto da Universidade Federal de Lavras —, a disciplina de POO exige uma análise que transcenda o veredito de “sucesso” ou “falha” na execução. Embora sistemas de avaliação automática sejam eficazes para validar a lógica de algoritmos lineares, eles apresentam limitações pedagógicas ao não contemplar a semântica e os princípios de *design* do código produzido pelo estudante, o que pode mascarar lacunas na compreensão do paradigma de objetos.

2.2 Correção Automática de Código

Historicamente, o *feedback* automatizado em ambientes educacionais tem se concentrado em aspectos quantitativos, como a correção de respostas objetivas ou a validação de código por meio de testes unitários. Embora essas abordagens promovam agilidade, elas limitam-se a informar se um resultado está correto ou incorreto, falhando em fornecer a orientação pedagógica necessária para que o discente compreenda a origem do seu erro conceitual.

Essa limitação é evidenciada por Keuning, Jeuring e Heeren (2018), que, ao realizarem uma revisão sistemática de 101 ferramentas de avaliação automática, constataram que a grande maioria foca estritamente na identificação de erros. Em contrapartida, são escassos os sistemas que oferecem orientações sobre como o aluno deve proceder ou que fornecem explicações sobre os conceitos teóricos envolvidos. Esse cenário indica uma falta de suporte ao processo de aprendizagem e à autonomia do aluno. Isso ocorre porque as ferramentas focam no resultado imediato do exercício, em vez de priorizar a formação de uma base teórica sólida.

Com o advento das LLMs, torna-se possível realizar uma análise semântica e qualitativa que atinja um nível de *feedback* que se aproxima do oferecido por um tutor humano. A viabilidade técnica dessa evolução é evidenciada por estudos como o de Wang (2023), que propôs um modelo especializado para geração automática de feedback em exercícios de Programação

Orientada a Objetos. Em seus experimentos, o sistema alcançou elevado grau de concordância com avaliações realizadas por especialistas, demonstrando o potencial da inteligência artificial para apoiar o processo de ensino de programação.

Diferentemente da proposta de Wang, que utiliza um modelo treinado especificamente para esse domínio, o presente trabalho investiga o uso de Grandes Modelos de Linguagem de propósito geral, explorando estratégias de contextualização e engenharia de *prompt* para produzir feedbacks pedagógicos em exercícios de Programação Orientada a Objetos.

2.3 Grandes Modelos de Linguagem (LLMs)

As LLMs são sistemas de inteligência artificial baseados em redes neurais profundas, projetados para compreender, gerar e manipular textos em linguagem natural ou código de programação. O termo “Grande Escala” refere-se tanto ao imenso volume de dados utilizados no treinamento quanto ao número de parâmetros da rede — chegando a centenas de bilhões —, o que permite ao modelo capturar nuances complexas de contexto, semântica e sintaxe.

A base tecnológica das LLMs modernas é a arquitetura *Transformer*, introduzida por Vaswani *et al.* (2017). O diferencial dessa arquitetura é o mecanismo de atenção (*attention*), que permite ao modelo processar todas as partes de uma sequência de texto simultaneamente. Diferente de modelos anteriores que liam o texto de forma linear, o *Transformer* consegue identificar quais palavras ou símbolos são mais relevantes entre si, independentemente da distância no texto. Em programação, essa capacidade é vital, pois permite que a IA compreenda como uma declaração de classe na linha inicial influencia um método implementado centenas de linhas abaixo.

De acordo com Bommasani *et al.* (2021), esses sistemas são classificados como “modelos de fundação”, pois são treinados em bases de dados massivas e diversificadas, adquirindo uma capacidade de generalização que permite a execução de tarefas para as quais não foram explicitamente programados. No contexto educacional, essas características permitem que as LLMs sejam utilizadas em tarefas que vão além da simples identificação de erros de compilação, incluindo a geração de explicações, sugestões de melhoria e análises qualitativas do código produzido pelos estudantes.

Dessa forma, as LLMs buscam contornar as limitações das ferramentas tradicionais de correção ao oferecerem uma análise qualitativa. Elas deixam de atuar como simples validadores

de “sucesso ou falha” para funcionarem como ferramentas capazes de gerar *feedback* contextualizado e mais próximo do acompanhamento realizado por um professor.

Por outro lado, o uso dessas ferramentas traz desafios técnicos causados pela natureza probabilística dos modelos. O principal deles é a ocorrência de alucinações, um fenômeno em que a inteligência artificial gera informações incorretas ou inventa dados que aparentam ser plausíveis e corretos. Embora esses sistemas consigam representar relações complexas entre conceitos e informações, eles geram respostas prevendo sequências linguisticamente aceitáveis com base em padrões aprendidos no treinamento, não possuindo uma validação absoluta sobre os dados nem um mecanismo interno de verificação da verdade. Por isso, tornam-se propensos a produzir informações falsas que apresentam uma estrutura formalmente coerente. Essas limitações tornam necessário o uso de técnicas de engenharia de *prompt* e de mecanismos externos de controle para possibilitar a consistência das respostas geradas.

2.4 IA como Tutor no Processo de Aprendizagem

De acordo com Pereira e Mello (2025), o uso de métodos manuais de avaliação em cursos de larga escala frequentemente resulta em atrasos na correção, o que impacta negativamente a motivação e a participação dos discentes. Os autores defendem que as LLMs oferecem uma solução para este problema ao possibilitarem respostas em tempo real, permitindo que o retorno pedagógico seja imediato e relevante para o processo de aprendizagem.

No entanto, um dos maiores desafios pedagógicos reside em evitar que a IA forneça a resposta pronta, comprometendo o esforço cognitivo do aluno. Jacobs e Jaschke (2024) demonstram que, ao utilizar modelos avançados como o GPT-4, é possível mitigar quase integralmente essa limitação, fornecendo sugestões e orientações sem expor a solução final. Os autores ressaltam que, para manter essa eficácia, é vital evitar o envio de informações contraditórias à IA. Um exemplo desse problema ocorre quando o sistema utiliza um gabarito correspondente a uma versão diferente do exercício submetido pelo estudante, induzindo o modelo a interpretar como incorretas soluções que, na realidade, atendem aos requisitos da atividade. Esse tipo de situação reforça a necessidade de fluxos de dados controlados para preservar a consistência do papel desempenhado pela IA como tutor.

2.5 Tutoria Inteligente e Estratégias de *Scaffolding*

No campo da tutoria pedagógica, destacam-se abordagens que buscam equilibrar o suporte imediato com a integridade acadêmica. Liffiton *et al.* (2023) propuseram o *CodeHelp*, um sistema que utiliza *guardrails* técnicos para inibir a entrega de soluções prontas. A arquitetura da ferramenta diferencia-se por um fluxo de processamento que prioriza a autonomia do discente através de uma estratégia de entrada estruturada e uma etapa de verificação de suficiência, na qual a IA avalia se a dúvida submetida possui contexto adequado; caso contrário, o tutor solicita clarificações antes de prosseguir com o auxílio. O *CodeHelp* também emprega um *pipeline* de múltiplos estágios para remover blocos de código das respostas, priorizando explicações textuais que guiem o aluno na resolução de erros de forma autônoma. Um diferencial relevante é a possibilidade de configuração de um conjunto de funções ou conceitos específicos que a IA não deve sugerir, possibilitando que o *feedback* esteja alinhado ao nível de conhecimento da turma.

De forma análoga, Sunil e Thakkar (2025) apresentam o *SocraticAI*, que fundamenta a interação em diálogos socráticos e no conceito de *scaffolding*. O sistema impõe restrições estruturadas, como limites de uso e a obrigatoriedade de o aluno articular suas tentativas prévias antes de receber auxílio. Ambas as ferramentas validam a premissa de que a IA deve atuar como um mediador cognitivo, promovendo a decomposição de problemas e a reflexão metacognitiva, princípios que norteiam a lógica de *feedback* deste trabalho.

Tecnicamente, o *SocraticAI* utiliza a geração aumentada por recuperação (*RAG*) para ancorar as respostas da IA exclusivamente no material didático da disciplina, assegurando o alinhamento curricular e minimizando alucinações. O sistema também impõe limites diários de uso para desencorajar consultas impulsivas e fomentar a formulação deliberada de perguntas. Os resultados demonstraram que essa imposição de restrições técnicas auxilia os estudantes a progredir de pedidos de ajuda vagos para consultas sofisticadas e orientadas à decomposição de problemas, promovendo não apenas a competência técnica, mas a maturidade cognitiva no aprendizado de computação.

3 METODOLOGIA

Neste capítulo, descrevem-se os procedimentos, técnicas e materiais utilizados para o desenvolvimento e validação do sistema de correção automatizada.

3.1 Identificação da Pesquisa

A presente pesquisa classifica-se como um estudo aplicado de natureza tecnológica. Quanto aos objetivos, possui caráter exploratório e experimental, visando investigar a utilização de modelos de linguagem na geração de *feedback* para o ensino de computação.

3.2 Contexto da Investigação

O contexto da investigação reside nas disciplinas GAC106 e GAC125, que tratam de conceitos e práticas de POO, vinculadas ao Departamento de Computação Aplicada (DAC-ICET) da Universidade Federal de Lavras (UFLA). Utilizaram-se submissões de códigos reais implementados em Java de discentes, anonimizadas, como base para os experimentos. A escolha por este ambiente justifica-se pela necessidade de avaliar a eficácia do sistema em um cenário acadêmico autêntico, onde a diversidade de soluções é representativa dos desafios reais enfrentados pelos estudantes.

3.3 O Problema da “Correção Estrutural Binária”

Tradicionalmente, sistemas de análise automática de código (como o *Dredd*) ou *frameworks* de testes unitários (como o *JUnit*), operam sob uma lógica de validação funcional baseada em casos de teste. Nesses modelos, cada caso de teste individual possui um veredito binário, verificando estritamente se a saída gerada pelo programa corresponde ao valor esperado para uma determinada entrada. Embora o resultado final de um exercício possa ser composto pelo sucesso em múltiplos testes, refletindo uma nota proporcional, a avaliação restringe-se ao comportamento externo do programa.

Embora eficazes para validar a funcionalidade, tais abordagens tratam apenas da validação de entrada e saída do programa e falham em identificar problemas na forma de construção

do programa (muito importantes em POO, como a falta de encapsulamento, por exemplo) ou em oferecer pistas pedagógicas que auxiliem o estudante na construção do conhecimento. A partir dessa percepção, o presente projeto não busca apenas automatizar a atribuição de notas, mas fornecer uma análise qualitativa mais próxima daquela realizada por um instrutor durante o processo de correção.

Visando solucionar a limitação das avaliações baseadas estritamente em resultados de execução, estabelece-se a proposta central deste sistema: utilizar LLMs integrados a um fluxo baseado em grafos para realizar uma análise semântica do código submetido. Essa abordagem permite comparar o código do aluno com a estrutura esperada pelo professor, gerando um *feedback* mais detalhado do que uma simples validação por testes.

3.4 Procedimentos Metodológicos

A condução desta pesquisa ocorreu de forma iterativa, seguindo um processo de prototipação e refinamento contínuo do sistema. Diferentemente de uma sequência linear de desenvolvimento, as etapas de implementação, experimentação e avaliação foram repetidas diversas vezes ao longo do projeto, permitindo a identificação de limitações e a incorporação de melhorias progressivas na arquitetura e nos mecanismos de geração de *feedback*.

As principais fases da pesquisa foram:

1. **Levantamento do Problema e Definição do Escopo:** Foi realizada a caracterização do problema da correção automática em disciplinas de Programação Orientada a Objetos, bem como a definição dos requisitos funcionais e pedagógicos do sistema proposto.
2. **Projeto e Implementação do Protótipo Inicial:** Desenvolveu-se uma primeira versão do sistema utilizando Python, LangGraph e integração com modelos de linguagem, estabelecendo a arquitetura básica de processamento e geração de *feedback*.
3. **Seleção dos Exercícios e Preparação dos Dados:** Foram selecionadas submissões reais de exercícios das atividades “Nave” e “Casa”, previamente anonimizadas. Também foram implementadas rotinas de pré-processamento para normalização dos códigos-fonte.
4. **Experimentação e Avaliação Iterativa:** O protótipo foi submetido a sucessivos ciclos de testes utilizando diferentes estratégias de *prompt* e configurações de processamento.

Os resultados obtidos foram analisados qualitativamente para identificar limitações, inconsistências e oportunidades de melhoria.

5. **Refinamento da Arquitetura e dos Prompts:** Com base nos problemas observados durante os experimentos, foram realizadas modificações na arquitetura do sistema e nos mecanismos de geração de *feedback*, buscando reduzir alucinações e aumentar a qualidade pedagógica das respostas.
6. **Validação Final:** Após a estabilização do protótipo, realizou-se uma avaliação final com foco na análise da qualidade dos *feedbacks* gerados pelo sistema. Foram observados aspectos relacionados à redução de alucinações, à diminuição de avaliações excessivamente rígidas e à capacidade do modelo de produzir diagnósticos mais alinhados aos objetivos pedagógicos da disciplina.

3.5 Ecossistema Tecnológico e Configuração do Ambiente

A arquitetura do projeto é fundamentada na interoperabilidade entre duas linguagens distintas: o Java e o Python. O Java foi selecionado como a linguagem de estudo dos discentes por ser o padrão das disciplinas de POO na UFLA e por seu rigor sintático. Em contrapartida, o motor de análise (*back-end*) é desenvolvido em *Python*, utilizando a biblioteca *LangGraph*. A escolha por *Python* justifica-se pela maturidade de seu ecossistema para Inteligência Artificial e pela facilidade de manipulação de *prompts*, permitindo uma integração fluída com APIs de LLMs.

Para a análise dos códigos e geração dos *feedbacks*, optou-se pela utilização do modelo **Gemini 2.5 Flash** via API. A escolha do modelo específico justifica-se pelo equilíbrio entre o baixo tempo de latência e a alta eficiência no processamento de contextos extensos, característica essencial para analisar múltiplos arquivos de código-fonte simultaneamente. A utilização de uma infraestrutura baseada em nuvem via API foi preferível à execução de modelos locais (como o Llama 3 via Ollama) para possibilitar que o sistema mantenha um desempenho consistente independentemente do *hardware* local, além de permitir analisar uma quantidade maior de código simultaneamente.

Para a implementação do protótipo e condução dos experimentos, utilizou-se o ambiente de desenvolvimento integrado (IDE) **Visual Studio Code (VSCode)**, escolhido por sua exten-

sibilidade e suporte nativo ao Python, facilitando a gestão de ambientes virtuais e a depuração de fluxos de agentes. A lógica de processamento do sistema é implementada por meio da biblioteca **LangGraph**, que permite representar as etapas de análise e geração de *feedback* como um fluxo estruturado de processamento.

Adicionalmente, o ambiente conta com o sistema de controle de versões **Git**, assegurando a rastreabilidade das alterações e a reprodutibilidade dos testes. A integração com as LLMs ocorre por meio de chamadas de API, permitindo que o motor em *Python* envie o código-fonte pré-processado e receba as inferências pedagógicas prontas, as quais são tratadas e apresentadas ao discente por meio de uma interface gráfica amigável desenvolvida em *Tkinter*.

4 DESENVOLVIMENTO DO SISTEMA

Neste capítulo, apresenta-se a implementação do sistema de correção automatizada e geração de *feedback* pedagógico. A seguir, apresentam-se a arquitetura, as tecnologias escolhidas, as principais funcionalidades, os desafios enfrentados e os testes realizados.

4.1 Arquitetura do Sistema

O sistema foi organizado em módulos, cada um com uma função bem definida. Isso facilita a manutenção e permite que cada parte evolua de forma independente. A seguir, estão os principais módulos e suas responsabilidades:

- **Interface do Usuário (`views/ui.py`):** Camada responsável pela captura de parâmetros informados pelo usuário, gerenciando componentes de seleção de enunciados, códigos de gabarito, diretórios de submissões de estudantes e conceitos pedagógicos específicos.
- **Módulo de I/O e Persistência (`utils/io.py`):** Subsistema encarregado da manipulação de arquivos no ambiente local. Realiza a leitura dos dados textuais, a descompactação em tempo de execução de submissões em formato `.zip`, a varredura e contagem indexada de arquivos com extensão `.java` e a gravação de logs estruturados de auditoria.
- **Módulo de Pré-processamento (`utils/formatters.py`):** Camada responsável pela sanitização e normalização do código-fonte, aplicando algoritmos de remoção de comentários e rotinas de concatenação dos arquivos de código.
- **Gerador de Prompts (`controllers/prompts.py`):** Módulo responsável por construir o contexto enviado ao modelo de linguagem. Combina as instruções pedagógicas do sistema com o enunciado do exercício, o gabarito de referência, os conceitos de POO selecionados pelo docente e o código submetido pelo estudante, produzindo a entrada utilizada durante o processo de avaliação.
- **Adaptador LLM (`controllers/llm.py`):** Componente que centraliza as interfaces de comunicação com o provedor de inteligência artificial, implementando políticas de controle de requisições e validação estruturada de saída. Na implementação atual, foca

no ecossistema integrado ao modelo adotado, estando estruturalmente preparado para o suporte multi-provedor nativo das bibliotecas de suporte.

- **Apresentação e Relatórios (`views/relatorio_html.py`):** Subsistema encarregado de transformar a resposta estruturada em um relatório visual em formato HTML, preservando a sintaxe de blocos de código e renderizando marcações textuais.
- **Orquestrador Central (`corretor_java.py`):** O núcleo funcional da aplicação, responsável por gerenciar o fluxo principal e coordenar os estados de transição do sistema por meio da biblioteca *LangGraph*.

4.1.1 Estrutura de Diretórios e Módulos

A organização física e a distribuição das responsabilidades dentro do ambiente de desenvolvimento Python seguem a árvore de arquivos detalhada a seguir:

```
01-corretor/
|__ corretor_java.py          # Orquestrador central (Ponto de entrada)

|__ views/                   # Camada de Apresentação e Visualização
    |__ ui.py                 # Interface gráfica (Tkinter)
    |__ relatorio_html.py     # Geração do relatório final em HTML

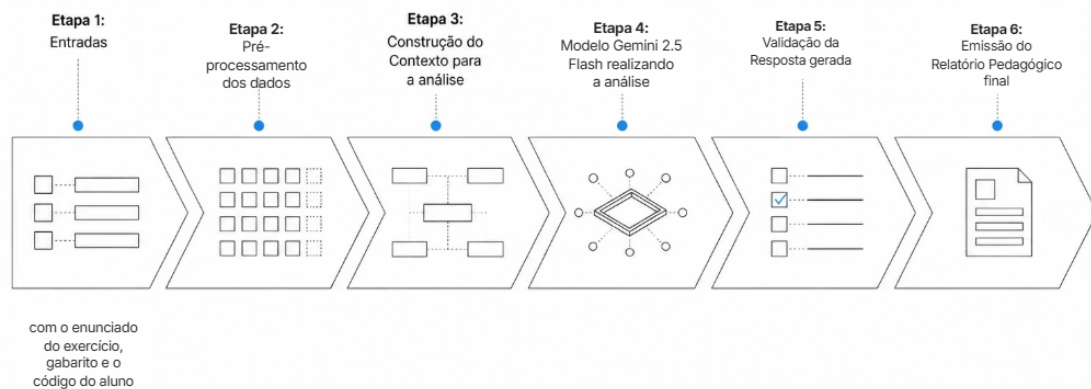
|__ controllers/             # Camada de Controle e Integrações
    |__ prompts.py            # PromptManager e injeção de contexto
    |__ llm.py                # Comunicação com o modelo cognitivo

|__ utils/                   # Camada de Serviços Gerais e Suporte
    |__ io.py                 # Manipulação de arquivos locais e logs
    |__ formatters.py         # Filtros de código e higienização estática
```


4.1.2 Fluxo Geral de Funcionamento do Sistema

A Figura 4.1 apresenta uma visão resumida do fluxo de funcionamento do protótipo desenvolvido. Embora o sistema seja apresentado ao usuário por meio de uma interface gráfica simplificada, internamente o processamento ocorre em etapas sequenciais que abrangem a preparação dos dados de entrada, a construção de contexto para análise, a geração do *feedback* pedagógico e a emissão do relatório final.

Figura 4.1 – Fluxo geral de funcionamento do sistema.



Fonte: Elaborada pelo autor.

Inicialmente, o sistema recebe como entrada o enunciado do exercício, o gabarito de referência e as submissões dos estudantes. Em seguida, os arquivos passam por uma etapa de pré-processamento responsável pela organização e normalização das informações que serão utilizadas durante a correção.

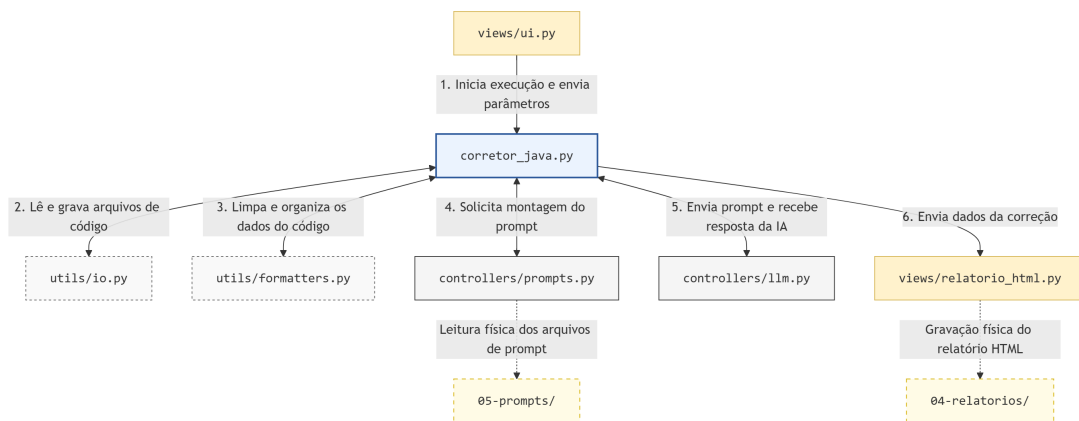
Posteriormente, ocorre a construção do contexto enviado ao modelo de linguagem, reunindo os elementos necessários para a avaliação da solução submetida. Com esse contexto estruturado, o modelo Gemini realiza a análise do código do estudante e produz uma avaliação acompanhada de *feedback* pedagógico.

As respostas geradas passam então por uma etapa de validação e organização estrutural, garantindo a consistência do formato de saída adotado pelo sistema. Por fim, é gerado automaticamente um relatório pedagógico em formato HTML, disponibilizando ao usuário os resultados da avaliação de forma estruturada e acessível.

4.1.3 Fluxo de Componentes

O relacionamento e o fluxo de chamadas a partir do núcleo funcional da aplicação encontram-se mapeados de forma esquemática na Figura 4.2, ilustrando o papel do componente principal como orquestrador centralizado das interações.

Figura 4.2 – Diagrama de blocos da arquitetura modular e fluxo centralizado de componentes.



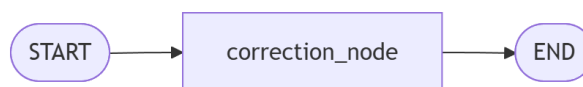
Fonte: Elaborada pelo autor.

4.1.4 Fluxo de Processamento no LangGraph

A lógica de execução do protótipo foi modelada utilizando o *framework* *LangGraph*. Na versão desenvolvida para esta pesquisa, o fluxo contém apenas um nó responsável pela correção do código submetido.

Do ponto de vista funcional, o sistema poderia ser implementado sem o uso do *LangGraph*. Contudo, a adoção desse framework permitiu manter uma arquitetura preparada para futuras extensões, possibilitando a incorporação de novas etapas de processamento sem alterações significativas na estrutura do sistema. A Figura 4.3 apresenta o fluxo utilizado no protótipo.

Figura 4.3 – Fluxo de estados implementado com LangGraph.



Fonte: Elaborada pelo autor.

O nó `correction_node` recebe o enunciado, o gabarito e o código do estudante, realiza a chamada ao modelo Gemini e retorna um objeto JSON estruturado contendo a avaliação e o *feedback* pedagógico.

4.2 Tecnologias e Ferramentas

As tecnologias foram escolhidas para proporcionar facilidade de uso, flexibilidade e bom suporte para Inteligência Artificial. Os principais critérios foram produtividade, integração com IA e facilidade de implantação. As escolhas técnicas são:

- **Linguagem de Programação:** Adotou-se o ambiente **Python 3.10** para o desenvolvimento do motor de análise, devido à sua alta produtividade nativa no tratamento de strings textuais e ampla oferta de bibliotecas voltadas para IA. O ecossistema **Java** permanece restrito ao escopo do estudante, servindo como o objeto de análise semântica.
- **Interface Gráfica (UI):** Utilizou-se a biblioteca nativa **tkinter**. A escolha fundamenta-se na eliminação de dependências externas pesadas, provendo uma interface gráfica (*GUI*) leve e multiplataforma para a seleção de diretórios.
- **Orquestração de Agentes:** Empregou-se o framework **langgraph**. A ferramenta permite modelar o processo de correção não de forma linear, mas como um grafo de estados cíclico, viabilizando rotinas futuras de auto-correção e refinamento de *prompts*.
- **Integração com LLM:** Utilizou-se o SDK oficial Google GenAI (**google-genai**) com foco no modelo *Gemini* (modelo configurável, ex.: *gemini-2.5-flash*).
- **Formatação Estética:** Adicionou-se suporte ao utilitário **google-java-format** via execução condicional de arquivo JAR externo. A ativação desse formatador é opcional e configurada de acordo com as variáveis de ambiente locais do sistema, buscando mitigar variações estéticas de indentação antes do processamento.
- **Bibliotecas Complementares:** Utilizou-se a biblioteca `python-dotenv` para o isolamento seguro de credenciais e chaves de API, e módulos integrados da biblioteca padrão como `zipfile` e `tempfile` para a manipulação temporária de submissões em memória. Adicionalmente, empregou-se a dependência `markdown` de forma opcional durante o fluxo de geração de relatórios para estruturar a renderização do HTML.

4.3 Funcionalidades Principais

O sistema foi pensado para cobrir todo o ciclo, desde o recebimento do exercício até a entrega do relatório ao estudante. As principais funcionalidades são:

1. **Seleção Dinâmica de Entradas:** Permite ao docente fornecer os caminhos locais do enunciado, das soluções de referência e das pastas contendo os exercícios entregues pelos discentes. O sistema gerencia tanto arquivos estruturados em diretórios tradicionais quanto submissões unificadas em formato comprimido `.zip`.
2. **Pré-processamento e Higiênização de Código:** O motor realiza a leitura, concatenação de múltiplos arquivos do mesmo discente e a remoção de comentários, reduzindo significativamente o consumo de *tokens* e eliminando ruídos textuais irrelevantes.
3. **Externalização e Auditoria de Prompts:** Toda a engenharia de instruções — composta pelas diretrizes de perfil (`persona.md`), restrições de escopo (`concepts.md`) e regras de correção (`system_prompt.md`) reside isolada no diretório `05-prompts/`. O sistema salva cópias físicas exatas de cada requisição montada no subdiretório `sent_prompts` por meio do `PromptManager`, enquanto as respostas brutas recebidas são persistidas em `received_responses` via `utils/io.save_feedback_json`, possibilitando a rastreabilidade completa do fluxo.
4. **Chamadas Resilientes e Tratamento de Respostas:** O adaptador foi projetado para assegurar que a comunicação com a API ocorra de forma consistente, minimizando interrupções durante o envio das requisições. Para lidar com as limitações de acessos gratuitos da API, que bloqueiam o sistema caso um volume elevado de requisições consecutivas seja atingido, implementou-se um mecanismo de repetição automática das requisições. Quando ocorre uma falha relacionada ao limite de uso da API, o sistema aguarda um intervalo de tempo antes de realizar uma nova tentativa, aumentando progressivamente esse tempo a cada falha consecutiva para evitar o bloqueio do serviço e garantir a continuidade do fluxo de execução.
5. **Renderização de Relatórios Didáticos:** Converte a estrutura de dados JSON validada em arquivos HTML responsivos com suporte a realce de sintaxe para o código Java e formatação visual fluida para as orientações do professor.

6. **Flexibilidade de Modos de Operação:** Suporte integrado ao Modo Único, focado em avaliações pontuais, e ao Modo Lote (*Batch*), projetado para iterar sequencialmente sobre turmas inteiras de estudantes.

4.4 Processo de Implementação e Desafios

O desenvolvimento do sistema foi conduzido inicialmente focado na construção de um protótipo funcional, priorizando a integração dos componentes e a validação do *pipeline* de correção.

Na primeira etapa, o foco concentrou-se na manipulação dos arquivos locais e na limpeza dos textos. Desenvolveu-se um analisador lexical customizado (*scanner* por caracteres) em `utils/formatters.py` para processar o código-fonte Java. Diferente de um filtro por expressões regulares simples, este analisador trata de forma precisa *strings*, *chars*, comentários de linha e blocos, preservando os literais do programa e removendo as linhas vazias apenas ao final do processo. Essa estratégia evitou o desperdício de *tokens* nas chamadas da API sem corromper a estrutura do código.

Com as rotinas de leitura funcionando, a segunda etapa consistiu em montar a estrutura de inteligência artificial utilizando o *LangGraph* para gerenciar o fluxo através de um grafo de estados e o SDK oficial `google.genai` para estabelecer a comunicação com o modelo *Gemini 2.5 Flash*. Para guiar as respostas da IA, utilizou-se a classe `PromptManager` em `controllers/prompts.py` apenas para a montagem dos *templates* e instruções do sistema. O papel de solicitar o `response_schema` e validar o JSON estruturado ficou a cargo da camada do adaptador LLM (`controllers/llm.py`), que integra diretamente o SDK `google.genai` (Gemini).

A terceira etapa foi o desenvolvimento da interface gráfica e das regras de salvamento em disco. Utilizou-se a biblioteca `tkinter` para desenho das telas e dos cartões visuais de seleção. Também foram adicionados botões para o professor decidir se os relatórios finais devem ser salvos em uma pasta centralizada ou de forma distribuída na pasta de cada aluno. Adicionalmente, a interface disponibiliza seletores para configurar a estratégia de retorno pedagógico adotada pelo sistema. Essa configuração é utilizada durante a montagem do contexto enviado ao modelo de linguagem, permitindo que os *feedbacks* sejam direcionados tanto para a apresen-

tação de uma solução completa quanto para o fornecimento de pistas baseadas na estratégia de *scaffolding*.

Ao testar o protótipo com submissões reais de alunos, surgiram quatro desafios práticos no código:

- **Organização de arquivos compactados:** Envolveu o gerenciamento dos relatórios gerados após o processamento de arquivos compactados em formato `.zip`. Para evitar que os relatórios finais fossem perdidos ou salvos dentro dos diretórios voláteis do sistema, desenvolveu-se o dicionário `EXTRACTED_SUBMISSIONS` em `utils/io.py`. Esse registro mapeia as pastas temporárias criadas durante a extração de volta ao caminho original do envio do discente, possibilitando que cada relatório em HTML seja copiado e persistido diretamente na pasta definitiva do respectivo estudante.
- **Alucinações sintáticas por formatação:** O modelo apresentou vulnerabilidades de interpretação quando o código do aluno apresentava uma formatação muito ruim, como chaves `{}` empilhadas ou linhas desalinhadas. Para mitigar essa vulnerabilidade, implementou-se uma chamada opcional ao utilitário `google-java-format` via arquivo JAR. O sistema tenta localizar o arquivo externo através da variável de ambiente `GOOGLE_JAVA_FORMAT_JAR` ou no caminho padrão `../06-tools/google-java-format.jar`; caso o JAR exista, o código é formatado antes do envio, e caso contrário, o motor prossegue utilizando o código original enviado pelo discente.
- **Refinamento do escopo de avaliação:** Durante os testes iniciais, observou-se que o modelo ocasionalmente produzia *feedbacks* excessivamente rígidos, atribuindo importância a aspectos que não faziam parte dos objetivos pedagógicos definidos para o exercício. Para alinhar a análise aos conceitos efetivamente avaliados, realizou-se um processo iterativo de refinamento dos arquivos de instrução utilizados pelo sistema. Utilizou-se o `system_prompt.md` para definir regras gerais de comportamento e formatação das respostas, o `persona.md` para orientar o modelo a atuar como um professor de Programação Orientada a Objetos durante o processo de correção, e o `concepts.md` para delimitar os conceitos de Programação Orientada a Objetos que deveriam ser priorizados durante a correção. Dessa forma, o modelo passou a concentrar sua avaliação em aspectos estruturais e conceituais da solução, reduzindo a ocorrência de observações fora do escopo da atividade.

- **Limitações de cota (Free Tier):** Envolveu o gerenciamento das limitações de cota do plano gratuito da API, onde o limite estrito de solicitações por minuto acionava erros. Para contornar isso em tempo de execução, implementou-se uma lógica de *Exponential Backoff* na camada do adaptador LLM, aplicando atrasos progressivos automáticos ao detectar quedas por limite de taxa. Como alternativa futura para eliminar a dependência de requisições em nuvem, aponta-se a viabilidade de rodar um modelo de linguagem de forma estritamente local (como o *Llama 3* via *Ollama*), dando autonomia total ao sistema dependendo apenas do hardware disponível.

4.5 Interface Gráfica e Relatórios do Sistema

Nesta seção, são descritos e apresentados os artefatos visuais do protótipo que foram desenvolvidos para mediar a interação com o docente e estruturar a exibição do retorno pedagógico fornecido aos discentes.

4.5.1 Interface Gráfica do Usuário

Para facilitar o gerenciamento das avaliações, a interface desenvolvida em *Tkinter* organiza os parâmetros de entrada de forma visual. O painel principal permite ao professor selecionar, em uma única tela, o enunciado do exercício, o gabarito de referência e a pasta contendo as submissões dos estudantes. Adicionalmente, a interface disponibiliza controles para a parametrização do comportamento do sistema, permitindo que o docente configure se o relatório final deve ser salvo de forma centralizada ou distribuído diretamente na pasta de submissão de cada discente, além de possibilitar a escolha do modo de retorno pedagógico entre a entrega direta da resposta ou apenas dicas. A disposição geral do painel completo encontra-se ilustrada na Figura 4.4.

Figura 4.4 – Painel de controle da interface gráfica para parametrização de submissões.

The image shows a web application window titled "Corretor de Projetos POO". The interface is divided into four main sections, each with a "Selecionar" button and a status indicator "Nenhum selecionado".

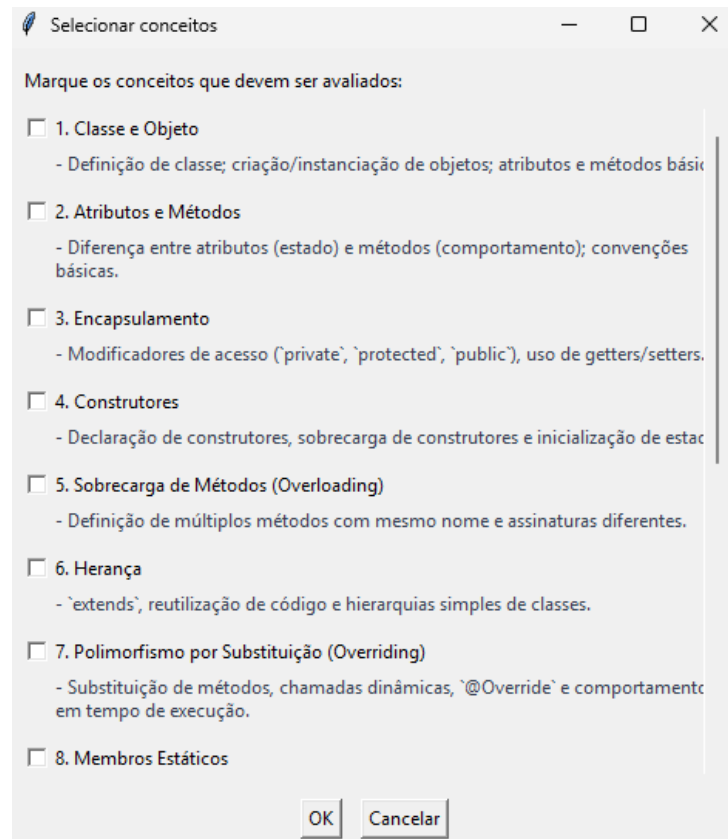
- 1. Enunciado**: Instruções (Markdown ou Texto)
- 2. Gabarito**: Pasta com arquivos .java (Opcional)
- 3. Código do Aluno**: Pasta com o projeto .java do aluno
- 4. Conceitos a Avaliar**: Escolher quais conceitos incluir na avaliação (Opcional)

Below these sections, there is a checkbox labeled "Salvar relatório na pasta do aluno" which is checked. Underneath, the "Modo de retorno:" is set to "Dar resposta" (selected with a radio button) instead of "Apenas dicas". At the bottom, there is a large green button labeled "EXECUTAR ANÁLISE".

Fonte: Elaborada pelo autor.

O componente específico voltado para a seleção e priorização dos conceitos de POO que guiarão a avaliação do modelo de linguagem é apresentado em detalhes na Figura 4.5. Esse seletor é preenchido de forma dinâmica a partir da leitura do arquivo `concepts.md`, exibindo na interface o título do conceito (como *4. Construtores*) acompanhado de sua respectiva descrição textual.

Figura 4.5 – Detalhe da interface gráfica para seleção e priorização de conceitos de POO.

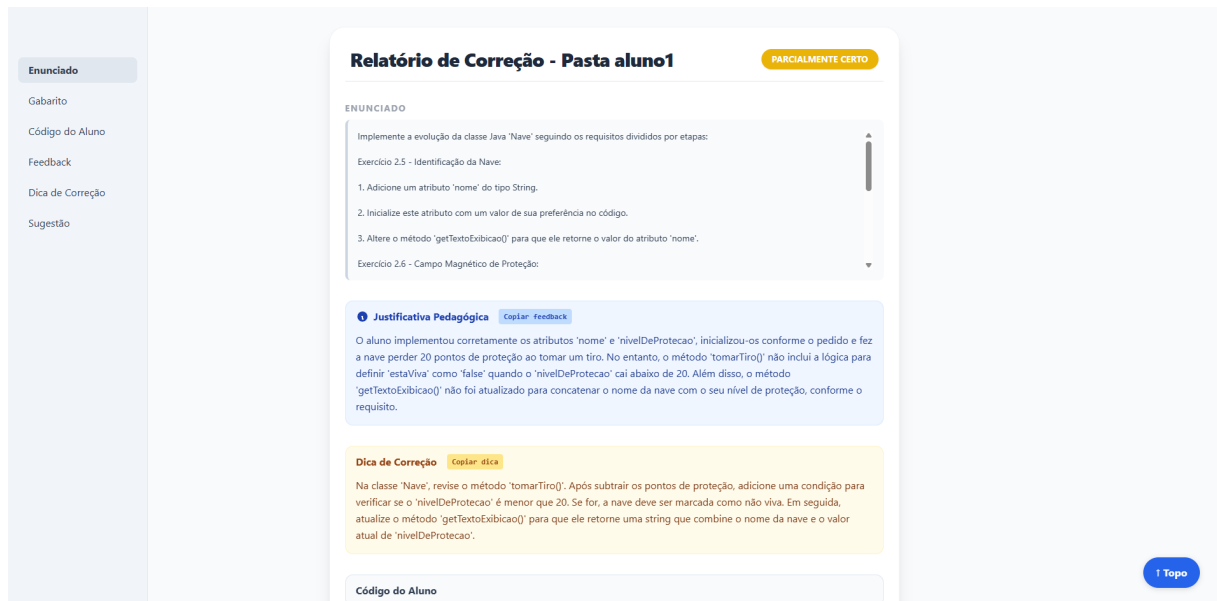


Fonte: Elaborada pelo autor.

4.5.2 Estrutura do Relatório Pedagógico Gerado

Após o processamento e a validação estruturada dos dados pelo orquestrador em *Python*, o sistema gera de forma automatizada um relatório didático em formato HTML. A interface gráfica resultante encontra-se apresentada nas Figuras 4.6 e 4.7.

Figura 4.6 – Interface gráfica do relatório pedagógico em HTML (Parte Superior).



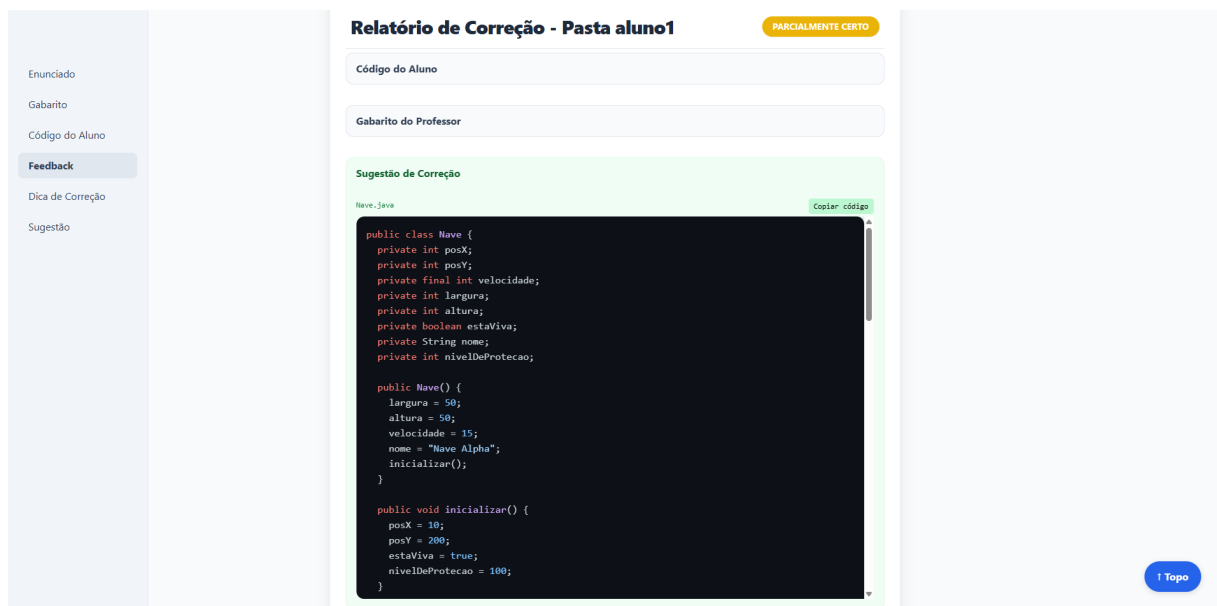
Fonte: Elaborada pelo autor.

A interface divide as informações em dois componentes principais: um menu lateral de navegação e uma área central para exibição do conteúdo. A estrutura inicial do relatório abrange os seguintes elementos:

1. **Menu Lateral de Navegação (Sidebar):** Fixado à esquerda, permite navegar diretamente entre as seções do documento (*Enunciado*, *Gabarito*, *Código do Aluno*, *Feedback*, *Dica de Correção* e *Sugestão*), funcionando como um sumário interativo;
2. **Cabeçalho de Status:** Localizado no topo, apresenta o título do documento e exibe o resultado da avaliação (como “*PARCIALMENTE CERTO*”) em destaque na cor amarela;
3. **Painel do Enunciado:** Caixa de texto com barra de rolagem interna que exibe as instruções e requisitos do exercício;
4. **Justificativa Pedagógica:** Bloco que renderiza a correção gerada pela IA, contendo também um botão para cópia rápida do texto (*Copiar feedback*);
5. **Dica de Correção:** Bloco que apresenta orientações complementares para auxiliar o estudante na resolução dos problemas identificados, oferecendo sugestões práticas para a adequação da implementação aos requisitos do exercício;

6. **Abas de Código:** Blocos colapsáveis dedicados à exibição das classes Java (Código do Aluno”, Gabarito do Professor” e “Sugestão de Correção”). Dependendo da configuração adotada pelo docente, determinadas seções podem ser omitidas do relatório final, como a exibição do gabarito ou da sugestão completa de correção. Por padrão, os painéis são carregados minimizados para priorizar a visualização das informações pedagógicas do relatório.

Figura 4.7 – Interface gráfica do relatório pedagógico em HTML (Parte Inferior).



Fonte: Elaborada pelo autor.

A continuação da interface, apresentada na Figura 4.7, evidencia a área destinada aos blocos colapsáveis de código e demais elementos complementares do relatório. Nessa região, são exibidas as seções correspondentes ao *Código do Aluno* e ao *Gabarito do Professor*, inicialmente minimizadas para favorecer a leitura das orientações pedagógicas. A figura também ilustra a exibição expandida do bloco de *Dica de Correção*, permitindo ao estudante consultar recomendações específicas para a melhoria da solução desenvolvida.

O ambiente conta ainda com um botão flutuante de rolagem automática (*Topo*), visível no canto inferior direito, que permite retornar instantaneamente ao início da página.

4.6 Procedimentos de Teste e Validação

A validação do protótipo durante a fase de desenvolvimento ocorreu de forma manual e experimental, focada em proporcionar estabilidade ao fluxo de controle e à correta integração entre os módulos do sistema. O objetivo principal foi verificar se o orquestrador em Python conseguia executar todas as etapas de manipulação de arquivos e chamadas de API sem interrupções.

Para os testes de funcionamento, utilizaram-se códigos Java reais das disciplinas de POO, correspondentes aos exercícios que serão apresentados e detalhados no Capítulo 5. Os testes manuais cobriram a validação das rotinas de extração de pacotes comprimidos em formato `.zip`, o processamento de projetos compostos por múltiplos arquivos (verificando se a concatenação de classes interdependentes ocorria sem perdas de texto) e o comportamento do adaptador diante de atrasos de rede ou respostas mal estruturadas pela API.

Nas situações em que o modelo gerava textos fora do padrão ou falhava no retorno estruturado, o adaptador realizava a impressão do conteúdo bruto na saída padrão do sistema (*stdout*) para fins de depuração imediata. Para evitar o travamento da aplicação diante de falhas de *parsing* do JSON, o fluxo principal do grafo de estados foi validado para acionar um mecanismo de *fallback*, gerando uma estrutura de dicionário padrão contendo a sinalização de erro nos campos avaliativos. Esta abordagem possibilitou a continuidade do motor para a geração do relatório final, preparando o ambiente para a condução dos experimentos científicos de avaliação de *feedback*, cujos estudos de caso detalhados, cenários de correção e análises qualitativas são apresentados e discutidos no Capítulo 5.

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta a análise dos experimentos realizados com o protótipo, avaliando o impacto da solução de referência (gabarito) na eficácia do *feedback* gerado. A análise foi estruturada em duas fases: testes controlados (Estudos de Caso I e II), utilizando a configuração inicial de *prompt*, e uma análise qualitativa de submissões reais que fundamentou o refinamento da estratégia pedagógica da IA.

5.1 Contextualização dos Cenários de Teste

Para a condução dos experimentos controlados e a subsequente validação do ecossistema de correção, utilizaram-se dois exercícios práticos adotados nas disciplinas de POO da instituição, denominados, respectivamente, “Nave” e “Casa”. Os enunciados completos dessas atividades encontram-se apresentados nos Anexos A e B. O cenário “Casa”, especificamente, é baseado nos projetos conceituais propostos por Barnes e Kölling (2008) na bibliografia de referência para o ensino da disciplina. Ambos os cenários foram projetados para que os discentes realizassem a evolução incremental de projetos de software preexistentes, aplicando conceitos estruturais sob diferentes níveis de complexidade.

O primeiro cenário, correspondente ao exercício “Nave”, concentrou-se na manipulação de estado, visibilidade de atributos e lógica condicional básica. Os estudantes receberam uma base de código de uma aplicação gráfica simplificada e foram instruídos a cumprir três etapas sequenciais de implementação:

1. **Identificação da Nave:** Inclusão de um atributo privado `nome` do tipo `String`, inicializado diretamente no código com um valor de preferência do discente, seguido pela alteração do método `getTextoExibicao()` para retornar o conteúdo desse novo atributo;
2. **Campo Magnético de Proteção:** Adição de um atributo inteiro `nivelDeProtecao` com inicialização fixa em 100 pontos. Considerando a lógica de decremento de 20 pontos a cada impacto sofrido, estabelecendo que a destruição da nave ocorreria apenas se o nível estivesse abaixo de 20 pontos no momento do dano, além de redefinir o valor para 100 pontos ao reiniciar o jogo;

3. **Feedback Visual:** Atualização do método `getTextoExibicao()` para apresentar de forma concatenada o nome atribuído à nave e o seu respectivo nível de proteção atual.

O segundo cenário, referente ao exercício “Casa”, demandou maior abstração ao incorporar conceitos de composição de objetos, modularização e orquestração de eventos múltiplos. A partir do código fornecido, o enunciado estabeleceu o desenvolvimento das seguintes regras:

1. **Composição e Novos Atributos:** Inclusão de um segundo atributo privado para um sol, chamado `sol2`, pertencente à classe `Circulo`, assegurando que ambas as instâncias de objetos fossem criadas e posicionadas corretamente na renderização da cena;
2. **Modularização:** Criação de um método público denominado `porDoSol()`, responsável por encapsular a lógica de movimentação descendente de ambos os objetos da classe `Circulo`;
3. **Interação entre Objetos:** Inserção de um atributo privado da classe `Pessoa` chamado `morador`, alterando a lógica para exibição do personagem na cena e condicionando o seu deslocamento em direção à residência estritamente após a conclusão do pôr do sol.

5.2 Primeira Estratégia de Prompt: Arquitetura Inicial e Premissas

Antes do início dos testes controlados, estabeleceu-se uma configuração inicial de instruções para o sistema, detalhada no Apêndice A como Primeira Estratégia de Prompt. O objetivo dessa versão inicial era validar a capacidade do ecossistema de correção em realizar uma análise híbrida, combinando a verificação estrutural do código-fonte com os requisitos pedagógicos solicitados.

A estrutura dessa primeira abordagem funcionava com a junção das instruções de persona e das regras rígidas do sistema no *system prompt*, que definia como a IA deveria processar a entrada e estruturar a saída em formato JSON. Esse bloco inicial recebia três entradas principais de contexto: o enunciado do exercício com as regras de negócio, o código-fonte submetido pelo estudante e a solução de referência idealizada pelo professor (gabarito). Quando o gabarito era fornecido, a instrução adicional de acoplamento (*Prompt*) era concatenada diretamente a esse conjunto de instruções do sistema.

A premissa central dessa estratégia consistia em fornecer à inteligência artificial o comportamento exato esperado do programa, reduzindo a margem de interpretação do modelo ao

determinar que a IA deveria considerar o gabarito como a única solução correta para o problema, realizando uma comparação direta e apontando as diferenças encontradas.

A intenção inicial era usar o gabarito como base principal para assegurar uma correção segura. O problema dessa abordagem foi fixar a solução de referência como regra absoluta, o que gerou um rigor excessivo e limitou a capacidade da IA de aceitar variações válidas no código dos estudantes.

5.3 Avaliação da Primeira Estratégia: Estudos de Caso e Diretrizes

Para a validação e análise do comportamento da Primeira Estratégia de Prompt, a avaliação do ecossistema de correção foi dividida em duas etapas experimentais. Inicialmente, realizou-se um teste controlado utilizando três perfis de submissões: uma solução 100% correta (em total conformidade com o gabarito), uma submissão não respondida (com o código original sem modificações) e soluções intermediárias (contendo erros lógicos e conceituais introduzidos intencionalmente). Posteriormente, o sistema foi submetido à análise de cenários extraídos de submissões reais de alunos das disciplinas de POO da instituição.

A experimentação inicial com os exercícios “Nave” e “Casa” utilizando essa base de dados permitiu observar como a ancoragem em um gabarito alterou a natureza da correção. Notou-se que, na ausência de uma solução de referência, a IA tendeu a atuar como um analisador funcional autônomo. Nesse estado, o modelo focou primordialmente na existência técnica dos componentes solicitados no enunciado e na identificação de erros de compilação.

Em contrapartida, a introdução do gabarito elevou o nível da avaliação. No cenário do exercício “Nave”, a presença da solução de referência permitiu que a IA identificasse não apenas a funcionalidade, mas a equivalência técnica. Por exemplo, o sistema pôde inferir que um código era idêntico à versão inicial não modificada, algo que uma análise sem gabarito interpretou apenas como ausência de implementação.

No exercício “Casa”, observou-se uma sofisticação maior no controle de estado (uso do booleano `porDoSolRealizado`). Com o suporte do gabarito, a IA demonstrou flexibilidade pedagógica ao classificar divergências paramétricas triviais — como pequenas variações de coordenadas — como aceitáveis, mantendo o foco na lógica de orquestração de eventos. Conforme discutido no referencial teórico, o foco residiu na arquitetura e não apenas na sintaxe, o que validou a abordagem híbrida proposta.

5.4 Análise Qualitativa e Limitações Identificadas

Após a validação controlada, o sistema processou códigos reais de discentes, revelando nuances críticas sobre o comportamento do modelo Gemini sob a primeira estratégia de *prompt*, discutidas nas subseções a seguir.

5.4.1 Consistência Pedagógica e Organização Estrutural do Código

Um dos resultados mais significativos foi a capacidade da IA de identificar violações de princípios de encapsulamento, modularização e separação de responsabilidades discutidos no referencial teórico. Observou-se que o modelo identificou casos de mistura de responsabilidades, como situações em que alunos implementaram a movimentação de um objeto (como o morador) dentro de métodos destinados ao controle do cenário, a exemplo da rotina de pôr do sol. Além disso, a IA demonstrou rigor na análise do encapsulamento, sugerindo a remoção de métodos *getters* desnecessários que não haviam sido solicitados, reforçando a importância da definição adequada das interfaces públicas dos objetos.

5.4.2 Limitações Técnicas e Rigor Excessivo

A análise qualitativa também revelou divergências indesejadas que comprometeram a experiência do usuário. Identificou-se uma vulnerabilidade à indentação, na qual o modelo alucinou erros de sintaxe (como classes não fechadas) devido a chaves desalinhadas no código do aluno. A partir dessa situação, passamos a considerar o uso de pré-processamento via linter para melhorar a previsibilidade do sistema.

Outro ponto crítico foi o denominado “Rigor Excessivo”. A IA tendeu a penalizar divergências estéticas (cores e tamanhos) que não eram restritas pelo enunciado. Esse comportamento evidenciou a necessidade de refinamento nas instruções de sistema, visando ensinar o modelo a diferenciar erros de arquitetura graves de escolhas criativas irrelevantes.

5.5 Evolução do Feedback: Segunda Estratégia de Prompt

Diante das limitações técnicas e do rigor excessivo observados na fase inicial, foi desenvolvida a Segunda Estratégia de Prompt (Apêndice B). Esta versão incorporou instruções explícitas de tolerância estética e reforçou a priorização de conceitos estruturais de Programação Orientada a Objetos em detrimento de detalhes visuais.

Como fonte complementar para o refinamento da segunda estratégia de *prompt*, foram consultados exemplos e recomendações de revisão de código presentes no repositório *GitHub Awesome Copilot*¹ (GitHub, 2026). Esse repositório reúne exemplos de engenharia de *prompts*, padrões de interação com modelos de linguagem e orientações para atividades de análise e revisão de código assistidas por IA. Os materiais disponibilizados serviram como referência para a elaboração de instruções voltadas à análise da lógica, da organização do código e dos aspectos estruturais das soluções avaliadas.

A reformulação foi validada qualitativamente a partir do reprocessamento dos cenários de teste, cujos resultados e reduções de falhas são detalhados a seguir.

Os testes com a estratégia refinada demonstraram uma redução da utilização de divergências estéticas e de implementação como critério de penalização das submissões. Em uma amostra de 18 entregas de estudantes utilizadas para comparação entre as duas estratégias, foi realizada uma análise manual dos *feedbacks* gerados. A abordagem inicial registrou 17 casos nos quais diferenças visuais ou de nomenclatura exerceram influência direta na justificativa da avaliação. Já na estratégia refinada, essas divergências passaram a desempenhar papel secundário nos *feedbacks* gerados. Embora ainda tenham sido observadas três menções a esse tipo de diferença, elas foram apresentadas apenas como observações complementares, sem impactar a classificação atribuída à submissão. Esses resultados indicam uma redução do comportamento de rigor excessivo identificado na primeira abordagem.

Embora essa comparação tenha sido realizada em uma amostra reduzida, os resultados motivaram a aplicação da estratégia refinada em um conjunto mais amplo de submissões, apresentado posteriormente na Subseção 5.5.1.

Para evidenciar essa evolução com dados práticos do projeto, identificou-se uma submissão real no exercício “Nave” que exemplifica as limitações da primeira abordagem e a correção

¹ Disponível em: <https://github.com/github/awesome-copilot>

obtida com o refinamento do sistema. Diante de uma mesma entrega, a primeira estratégia de *prompt* emitiu uma avaliação de “Parcialmente Certo”, apresentando a seguinte justificativa:

Contudo, há duas questões a serem observadas: (1) O método `getTextoExibicao()` do aluno não segue a formatação exata do exemplo fornecido no enunciado ('NomeDaNave - Proteção: Nivel'), apresentando apenas 'NomeDaNave Nivel'. (2) A classe 'Nave' no código do aluno apresenta um erro de sintaxe grave no fechamento de chaves do método `tomarTiro()`, o que resultaria em um erro de compilação, pois a classe 'Nave' não é fechada corretamente.

A análise desse retorno evidenciou as duas falhas de rigor discutidas anteriormente. A primeira consistiu na penalização estética pelo fato de o estudante omitir o hífen na concatenação de texto, embora a lógica estrutural estivesse correta. A segunda falha foi a alucinação de um erro de sintaxe relacionado ao fechamento de chaves, motivada exclusivamente por problemas de desalinhamento de indentação no código original do discente.

Ao submeter a mesma entrega para a segunda estratégia de *prompt* — após o pré-processamento e padronização estrutural via *linter google-java-format* —, o sistema mitigou os falsos positivos. Na abordagem anterior, uma chave (`}`) mal indentada levou o modelo a interpretar erroneamente que o bloco não havia sido fechado e que o código apresentaria erro de compilação, muito embora a sintaxe estivesse válida. Com a formatação do *linter*, o veredito foi corrigido para “Certo”, apresentando a seguinte resposta:

A implementação da classe 'Nave' está correta e atende a todos os requisitos dos exercícios 2.5, 2.6 e 2.7. O aluno adicionou e inicializou corretamente o atributo 'nome', implementou o atributo 'nivelDeProtecao' com a lógica de inicialização, decremento e condição de 'morte' da nave, e garantiu que a proteção é reiniciada. O método `getTextoExibicao()` foi atualizado para exibir tanto o nome quanto o nível de proteção de forma concatenada, conforme solicitado.

Esse cenário prático confirmou que a combinação do tratamento de formatação de código com instruções baseadas em tolerância visual reduziu o rigor excessivo, permitindo que a avaliação focasse nos conceitos de orientação a objetos e nas regras de negócio estabelecidas.

Da mesma forma, no cenário do exercício “Casa”, a análise de uma submissão real confirmou o comportamento de rigor excessivo da primeira estratégia e como o refinamento foi necessário para validar regras de negócio complexas. No caso analisado, o estudante realizou a evolução do cenário modificando os parâmetros de movimento, tamanho e cor do componente, nomeando-o como `segundoSol`. Além disso, o discente integrou a lógica de movimentação da

classe `Pessoa` diretamente dentro do método `porDoSol()`. Sob a primeira estratégia de *prompt*, a IA retornou o resultado de “Parcialmente Certo” com a seguinte análise:

O aluno demonstrou compreensão parcial dos requisitos do exercício. Para o Exercício 1.5, o atributo para o segundo sol foi adicionado e instanciado corretamente, porém, os parâmetros de cor, posição horizontal e tamanho para 'segundoSol' no método 'desenhar()' não corresponderam exatamente ao solicitado pelo gabarito (o gabarito esperava um sol amarelo, posicionado em '180' e com tamanho '60', enquanto o aluno utilizou azul, '190' e '80' respectivamente). No Exercício 1.7, o método 'porDoSol()' foi criado e a lógica de mover os sóis verticalmente para baixo foi implementada, mas a distância de movimento foi diferente da especificada no gabarito (300 em vez de 120). Além disso, o método 'porDoSol()' do aluno incluiu a lógica para mover a 'pessoa', o que viola o princípio de responsabilidade única e a ordem dos eventos. Faltou a introdução de uma variável de estado booleana para indicar que o pôr do sol ocorreu e para controlar a execução de ações dependentes, como a caminhada da pessoa. Para o Exercício 1.8, o atributo 'Pessoa' foi adicionado e a pessoa aparece na cena, mas com uma posição inicial diferente da do gabarito. O principal erro foi na implementação da interação: a lógica para o 'morador' caminhar em direção à casa não foi implementada em um método separado, como 'moradorCaminhaAteCasa()', e foi inserida diretamente no método 'porDoSol()'. Consequentemente, a condição de a caminhada ocorrer 'somente APÓS o pôr do sol ter ocorrido' não foi controlada adequadamente. Por fim, as cores dos novos objetos ('segundoSol' e 'pessoa') não foram atualizadas nos métodos 'definirPretoEBranco()' e 'definirColorida()', indicando uma falta de consistência na manutenção do estado dos objetos adicionados.

O retorno explicitou como a primeira configuração do sistema misturou a avaliação de falhas arquiteturais graves com divergências estéticas secundárias. O modelo identificou corretamente problemas estruturais reais, como a violação do princípio de responsabilidade única e a ausência do controle de estado booleano. Contudo, o sistema penalizou o estudante com o mesmo nível de rigor por escolher a cor azul para o sol ou por alterar as coordenadas de pixels de deslocamento.

Quando essa mesma submissão foi processada pela segunda estratégia de *prompt* — desenvolvida para ignorar variações visuais não essenciais —, o sistema isolou o rigor excessivo, concentrando a análise na arquitetura e na modularização do software. Embora o resultado te-

nha se mantido como “Parcialmente Certo” devido aos problemas estruturais, o sistema gerou a seguinte resposta:

O aluno demonstrou entendimento na adição e instanciação dos novos atributos para o segundo sol e o morador. O método 'porDoSol()' foi criado e move corretamente os dois sóis verticalmente. No entanto, a solução apresenta falhas cruciais de modularização e controle de fluxo, especialmente na implementação da interação do morador com a casa. A lógica de movimentação do morador foi incorretamente inserida dentro do método 'porDoSol()', violando o requisito de que o morador caminhe após o pôr do sol ter ocorrido (Exercício 1.8.3). Além disso, faltou um atributo de estado (como uma flag booleana) para controlar a sequência dos eventos e garantir que o morador só se mova no momento certo, e para impedir a execução repetida do pôr do sol. Houve também pequenas divergências de nomenclatura para os atributos (segundoSol em vez de sol2 e pessoa em vez de morador) e a falta de atualização dos novos objetos nos métodos definirPretoEBranco() e definirColorida(), o que causa inconsistência na aplicação de cores.

A comparação entre as duas saídas evidenciou uma mudança significativa no comportamento do avaliador automatizado, embora não tenha removido o comportamento de rigor completamente. Enquanto o primeiro *feedback* atuou de forma mais minuciosa sobre valores exatos e numéricos do gabarito (como pixels e cores específicas), a segunda abordagem mostrou-se mais conceitual e focada em *design* de software. O modelo refinado desconsiderou as coordenadas e cores alternativas para a renderização, mas manteve um critério rígido de conformidade técnica ao continuar penalizando pequenas divergências de nomenclatura de atributos e a falta de consistência na manutenção do estado dos objetos dentro dos métodos de configuração de cores.

A inclusão de ambos os cenários práticos permitiu constatar que a reengenharia das instruções reduziu a influência de detalhes superficiais na avaliação das submissões, conferindo ao ecossistema maior flexibilidade pedagógica sem abrir mão da análise de aspectos estruturais relevantes do código.

5.5.1 Avaliação Ampliada em Submissões Reais

Além dos estudos de caso apresentados anteriormente, a segunda estratégia de *prompt* foi aplicada a um conjunto mais amplo de submissões reais provenientes das disciplinas de

Programação Orientada a Objetos. O objetivo dessa etapa foi observar o comportamento do sistema em uma amostra mais representativa de implementações desenvolvidas pelos estudantes, permitindo identificar padrões recorrentes de erro e avaliar a consistência da abordagem refinada.

Ao todo, foram processadas 96 submissões distribuídas entre os exercícios “Nave” e “Casa”. A Tabela 5.1 apresenta a distribuição das classificações produzidas pelo sistema.

Tabela 5.1 – Distribuição das avaliações realizadas pela segunda estratégia de *prompt*

Exercício	Certo	Parcialmente Certo	Errado	Total
Nave	8	31	5	44
Casa	0	51	1	52
Total	8	82	6	96

Os valores apresentados na Tabela 5.1 correspondem às classificações produzidas automaticamente pelo sistema durante a execução dos experimentos, não constituindo uma comparação com uma avaliação de referência realizada por docentes.

A ausência de classificações “Certo” no exercício “Casa” pode estar relacionada à maior complexidade dos requisitos de modularização, controle de fluxo e coordenação entre objetos exigidos pela atividade, o que levou a maioria das submissões a apresentar ao menos alguma inconsistência estrutural identificada pelo sistema.

Observa-se uma predominância de classificações “Parcialmente Certo” em ambos os exercícios. Esse resultado sugere que a maioria dos estudantes conseguiu implementar parte significativa dos requisitos propostos, mas apresentou dificuldades relacionadas principalmente à modularização, ao controle de estado e à organização da lógica de negócio. Durante a análise das submissões, também foram identificados padrões recorrentes de erros associados à mistura de responsabilidades entre métodos, à ausência de mecanismos adequados de controle de fluxo e a divergências de nomenclatura em relação às especificações dos exercícios.

Embora as classificações “Certo” representem uma parcela minoritária da amostra, elas não devem ser interpretadas isoladamente, como um indicativo de desempenho insatisfatório dos estudantes. Em muitos casos, as submissões atendiam aos requisitos centrais dos exercícios, porém apresentavam pequenas inconsistências de modularização, nomenclatura ou organização estrutural, o que levou o sistema a classificá-las como “Parcialmente Certo”. Dessa forma, a

predominância dessa categoria reflete mais a natureza incremental do processo de aprendizagem em POO do que a ausência de compreensão dos conceitos abordados.

Esse comportamento pode estar relacionado ao fato de que os exercícios analisados pertencem às etapas iniciais da disciplina de Programação Orientada a Objetos. Nesse momento da formação, é comum que os estudantes ainda estejam em processo de transição de uma visão predominantemente procedural para uma abordagem baseada em encapsulamento, abstração e responsabilidade entre objetos. Como consequência, dificuldades envolvendo modularização, separação de responsabilidades e modelagem adequada de estados tendem a ocorrer com frequência, mesmo quando os requisitos funcionais principais são parcialmente atendidos.

Outro resultado relevante refere-se ao tratamento de divergências estéticas e de implementação que não comprometiam os requisitos funcionais dos exercícios. Durante a análise manual das justificativas geradas pelo sistema, foram identificados 28 casos nos quais diferenças em relação ao gabarito foram consideradas pedagogicamente irrelevantes. Nessas situações, o sistema registrou tais divergências apenas como observações complementares no *feedback*, concentrando a análise nos requisitos funcionais e estruturais da solução. No exercício “Casa”, foram observados 17 casos associados principalmente a variações de coordenadas geométricas, tamanhos de elementos gráficos, cores e posicionamento visual da cena. Já no exercício “Nave”, registraram-se 11 ocorrências relacionadas principalmente à nomenclatura de variáveis, à formatação textual e a pequenas diferenças na construção das mensagens exibidas ao usuário.

Esse resultado é compatível com a hipótese de que a estratégia refinada reduziu a rigidez observada na abordagem inicial, priorizando a avaliação da lógica de programação e dos conceitos de orientação a objetos em detrimento de aspectos meramente visuais da implementação. Entretanto, essa flexibilidade não ocorreu de forma irrestrita. Observou-se que o sistema continuou penalizando alterações quando estas comprometiam aspectos estruturais da solução, como violações de convenções fundamentais da linguagem Java, modificações em assinaturas de métodos exigidos pela arquitetura dos exercícios ou mudanças que produziam impactos funcionais no comportamento esperado da aplicação.

Além disso, verificou-se que o sistema ainda tende a mencionar parte dessas divergências em seus *feedbacks*, mesmo quando elas não influenciam a classificação final da solução. Tal comportamento indica uma oportunidade de aprimoramento futuro, visando tornar as de-

volutivas mais objetivas e concentradas exclusivamente nos aspectos pedagógicos e funcionais relevantes para a aprendizagem.

Os resultados obtidos indicam que a segunda estratégia de *prompt* tornou o processo de avaliação mais aderente aos objetivos pedagógicos da disciplina, reduzindo penalizações associadas a aspectos visuais e concentrando a análise em conceitos fundamentais de Programação Orientada a Objetos, como encapsulamento, modularização e responsabilidade entre objetos. Em conjunto com os estudos de caso apresentados anteriormente, essa avaliação ampliada fornece evidências de que o refinamento da engenharia de *prompt* contribuiu para tornar os *feedbacks* mais consistentes, interpretáveis e alinhados às necessidades do processo de aprendizagem, sem eliminar a capacidade do sistema de identificar problemas estruturais relevantes nas soluções avaliadas.

5.6 Limitações Técnicas e Operacionais do Protótipo

Apesar dos resultados satisfatórios observados durante os experimentos, o protótipo desenvolvido ainda apresenta limitações técnicas e operacionais que impedem sua adoção imediata como ferramenta de avaliação acadêmica em larga escala.

Uma das principais limitações está relacionada à natureza probabilística dos Grandes Modelos de Linguagem (LLMs). Devido ao comportamento não-determinístico intrínseco a esse tipo de modelo, mesmo com o refinamento das estratégias de *prompt* e com o uso de mecanismos de controle de contexto, o sistema permanece sujeito à ocorrência de alucinações, interpretações ambíguas e inconsistências entre inferências. Em tese, diante dessa característica, duas submissões semanticamente equivalentes estão sujeitas a receber *feedbacks* diferentes dependendo da formulação do *prompt*, da organização do código ou do estado contextual da inferência.

Outra limitação importante refere-se à dependência de serviços externos baseados em nuvem. A utilização do modelo Gemini via API introduz restrições relacionadas à disponibilidade do serviço, limites de requisição, latência de rede e custos operacionais. Durante os experimentos, observaram-se interrupções ocasionadas por limites de cota do plano gratuito, exigindo a implementação de estratégias de *Exponential Backoff* para evitar falhas consecutivas no fluxo de correção.

Do ponto de vista técnico, o sistema ainda não realiza análise semântica formal do código-fonte. Embora o modelo consiga inferir problemas estruturais e conceituais de POO, a validação continua baseada predominantemente em interpretação textual probabilística, e não em compilação semântica profunda. Dessa forma, o sistema pode deixar de detectar inconsistências específicas ou interpretar incorretamente determinadas construções sintáticas.

Outra limitação relevante está relacionada ao escopo dos cenários utilizados na validação experimental. Embora os exercícios “Nave” e “Casa” tenham sido selecionados por representarem diferentes níveis de complexidade e permitirem avaliar conceitos iniciais de Programação Orientada a Objetos, os experimentos foram conduzidos sobre apenas dois conjuntos de atividades da disciplina. Além disso, os exercícios analisados concentraram-se principalmente em conceitos como encapsulamento, composição, modularização e controle de estado, não contemplando outros tópicos relevantes do paradigma, como herança, polimorfismo, interfaces e classes abstratas. Dessa forma, a generalização dos resultados para todo o escopo da Programação Orientada a Objetos deve ser realizada com cautela.

Além disso, o protótipo foi validado exclusivamente no contexto de exercícios de Programação Orientada a Objetos em Java. Portanto, não é possível generalizar os resultados para outras linguagens de programação, paradigmas computacionais ou disciplinas que exijam critérios de avaliação distintos. A própria engenharia de *prompts* utilizada foi ajustada especificamente para os cenários “Nave” e “Casa”, o que indica uma dependência significativa do domínio educacional analisado.

No aspecto operacional, destaca-se que os experimentos foram conduzidos em um conjunto relativamente controlado de submissões. Embora tenham sido utilizados códigos reais de estudantes, o sistema ainda não foi submetido a cenários de larga escala, múltiplas turmas ou ambientes reais de integração institucional. Dessa forma, questões relacionadas à escalabilidade e desempenho sob carga permanecem em aberto.

Adicionalmente, o sistema não elimina a necessidade de supervisão docente. O *feedback* gerado pela IA deve ser interpretado como um mecanismo de apoio pedagógico e não como substituição definitiva da avaliação humana. Em especial, casos ambíguos, soluções criativas ou implementações semanticamente válidas, mas estruturalmente incomuns, ainda demandam validação por parte do professor para evitar avaliações injustas.

Por fim, destaca-se que o comportamento do sistema permanece altamente sensível à engenharia de *prompts*. Pequenas alterações nas instruções fornecidas ao modelo podem modi-

ficar significativamente o rigor, a tolerância e o estilo do *feedback* produzido. Essa característica evidencia que parte da qualidade da avaliação ainda depende fortemente da formulação manual das estratégias de orquestração e contextualização utilizadas no *pipeline*.

Essas limitações evidenciam que o protótipo deve ser compreendido como uma ferramenta experimental de apoio ao processo pedagógico, servindo como base para futuras pesquisas envolvendo validação semântica híbrida, modelos locais especializados, integração com compiladores e mecanismos mais robustos de avaliação educacional automatizada.

6 CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a avaliação de um protótipo para geração automática de *feedback* pedagógico em exercícios de Programação Orientada a Objetos, utilizando Grandes Modelos de Linguagem (LLMs) integrados a um ecossistema de processamento e análise de código-fonte. A proposta buscou investigar de que forma a combinação entre enunciados, soluções de referência e estratégias de engenharia de *prompt* poderia contribuir para a produção de avaliações mais úteis e alinhadas aos objetivos educacionais da disciplina.

Para alcançar esse objetivo, foi desenvolvido um sistema capaz de processar submissões de estudantes, organizar o contexto necessário para a correção e gerar relatórios pedagógicos estruturados. O ecossistema implementado incorporou mecanismos de pré-processamento, orquestração de contexto, integração com modelos de linguagem por meio de API e geração automática de relatórios em formato HTML.

Os experimentos realizados permitiram observar que a utilização de uma solução de referência contribui para ampliar a capacidade de análise do modelo, permitindo identificar não apenas a presença de funcionalidades, mas também aspectos relacionados à equivalência técnica e à organização estrutural do código. Entretanto, os resultados também evidenciaram que uma dependência excessiva do gabarito pode introduzir rigidez indevida no processo avaliativo, levando à penalização de diferenças que não comprometem os requisitos funcionais da atividade.

A partir dessas observações, foi desenvolvida uma segunda estratégia de *prompt*, voltada para reduzir o rigor excessivo e priorizar conceitos centrais de Programação Orientada a Objetos. Os resultados obtidos indicaram uma redução da influência de aspectos visuais na avaliação, permitindo que os *feedbacks* gerados se concentrassem com maior frequência em elementos como encapsulamento, modularização, controle de estado e responsabilidade entre objetos. Além disso, a aplicação da estratégia refinada em um conjunto ampliado de submissões reais demonstrou que o sistema foi capaz de identificar padrões recorrentes de dificuldades enfrentadas pelos estudantes, produzindo devolutivas mais consistentes e alinhadas aos objetivos pedagógicos da disciplina.

Apesar dos resultados promissores, o trabalho apresenta limitações relacionadas à natureza probabilística dos modelos de linguagem, à dependência de serviços externos, à ausência de análise semântica formal do código-fonte e ao escopo restrito dos cenários avaliados. Dessa

forma, os resultados obtidos não devem ser interpretados como evidência de substituição da avaliação docente, mas sim como indicativos do potencial da IA como ferramenta de apoio ao processo de ensino e aprendizagem.

Como trabalhos futuros, destacam-se a integração do sistema com analisadores sintáticos e mecanismos de compilação, permitindo verificar automaticamente se o código submetido compila ou não. Também se propõe incorporar uma análise individualizada para cada critério selecionado pelo usuário, possibilitando que conceitos como construtores, sobrecarga, método *main*, métodos estáticos, *setters* e *getters* sejam avaliados separadamente. Outra possibilidade é a geração automática de notas pela IA, associando os diagnósticos qualitativos a critérios avaliativos mais objetivos.

Por fim, permanecem como possibilidades futuras a comparação sistemática entre os *feedbacks* gerados pelo sistema e avaliações realizadas por docentes, a avaliação em outras linguagens de programação e disciplinas, e a investigação de modelos especializados executados localmente.

Conclui-se que a engenharia de *prompt*, associada a mecanismos adequados de contextualização e pré-processamento, exerce papel fundamental na qualidade dos *feedbacks* gerados por LLMs em cenários educacionais. Os resultados obtidos demonstram que, quando utilizadas de forma criteriosa, essas tecnologias possuem potencial para auxiliar docentes e estudantes, ampliando as possibilidades de acompanhamento e apoio ao aprendizado em Programação Orientada a Objetos.

REFERÊNCIAS

- ARMSTRONG, D. J. The quarks of object-oriented development. **Communications of the ACM**, ACM, v. 49, n. 2, p. 123–128, fev. 2006.
- BARNES, D. J.; KÖLLING, M. **Programação Orientada a Objetos com Java: uma introdução prática usando o BlueJ**. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
- BOMMASANI, R. *et al.* **On the Opportunities and Risks of Foundation Models**. Stanford, California, USA, 2021. Disponível em: <https://arxiv.org/abs/2108.07258>.
- BOOCH, G.; MAKSIMCHUK, R. A.; ENGLE, M. W. **Object-Oriented Analysis and Design with Applications**. 3. ed. Boston: Addison-Wesley, 2007.
- CIPRIANO, B. P.; ALVES, P. Gpt-3 vs object oriented programming assignments: An experience report. In: **Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2023)**. Turku, Finland: [s.n.], 2023. p. 61–67. Disponível em: <https://doi.org/10.1145/3587102.3588814>.
- ECKERDAL, A.; BERGLUND, A. What does it take to learn ‘programming thinking’? In: **Proceedings of the First International Workshop on Computing Education Research (ICER’05)**. Seattle, Washington, USA: ACM, 2005. p. 135–142.
- GitHub. **Awesome Copilot: A curated list of awesome Copilot resources, guides, and prompts**. 2026. Disponível em: <https://github.com/github/awesome-copilot>. Acesso em: 8 jun. 2026.
- JACOBS, S.; JASCHKE, S. Evaluating the application of large language models to generate feedback in programming education. In: **Proceedings of the 2024 IEEE Global Engineering Education Conference (EDUCON)**. Kos Island, Greece: IEEE, 2024. p. 1–7.
- KEUNING, H.; JEURING, J.; HEEREN, B. A systematic literature review of automated feedback generation for programming exercises. **ACM Transactions on Computing Education**, ACM, v. 19, n. 1, p. 3:1–3:43, set. 2018.
- LIFFITON, M. *et al.* **CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes**. [s.l.], 2023. Disponível em: <https://arxiv.org/abs/2308.06921>.
- PEREIRA, A. F.; MELLO, R. F. A systematic literature review on large language models applications in computer programming teaching evaluation process. **IEEE Access**, v. 13, p. 113449–113460, 2025. Disponível em: <https://ieeexplore.ieee.org/document/11058691>.
- SCHNEIDER, B. **Dredd - Juiz Online**. Lavras, 2021. Desenvolvido a partir do programa original (Algod) de Renato R. R. de Oliveira. Disponível em: <https://bruno.dac.ufla.br/aulas/ialg/dredd.html>. Acesso em: 8 jun. 2026.
- SUNIL, K.; THAKKAR, A. Socraticai: Transforming llms into guided cs tutors through scaffolded interaction. In: **Proceedings of the 2025 ACM India Joint International Conference on Compute (COMPUTE 2025)**. New Delhi, India: [s.n.], 2025. p. 1–6. Disponível em: <https://arxiv.org/abs/2512.03501>.

VASWANI, A. *et al.* Attention is all you need. In: **Advances in Neural Information Processing Systems (NeurIPS 2017)**. Long Beach, California, USA: [s.n.], 2017. v. 30, p. 5998–6008. Disponível em: <https://papers.nips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.

WANG, F. H. Efficient generation of text feedback in object-oriented programming education using cached performer revision. **Machine Learning with Applications**, v. 13, p. 100481, 2023.

ANEXOS

ANEXO A – ENUNCIADO DO EXERCÍCIO NAVE

Implemente a evolução da classe Java Nave seguindo os requisitos divididos por etapas:

Exercício 2.5 – Identificação da Nave

1. Adicione um atributo `nome` do tipo `String`.
2. Inicialize este atributo com um valor de sua preferência no código.
3. Altere o método `getTextoExibicao()` para que ele retorne o valor do atributo `nome`.

Exercício 2.6 – Campo Magnético de Proteção

1. Adicione um atributo `nivelDeProtecao` do tipo `int`.
2. A nave deve iniciar sempre com 100 pontos de proteção.
3. Implemente a lógica para que, ao receber um tiro, a nave perca 20 pontos de proteção.
4. A nave só deve morrer (ser destruída) se tomar um tiro e o seu campo de proteção estiver abaixo de 20.
5. Garanta que ao reiniciar o jogo, o nível de proteção volte para 100.

Exercício 2.7 – Feedback Visual

1. Atualize o método `getTextoExibicao()` para que ele exiba o nome da nave e o nível de proteção atual de forma concatenada.

“ Exemplo:

"Nave Alpha - Proteção: 80" “

ANEXO B – ENUNCIADO DO EXERCÍCIO CASA

Implemente a evolução da classe Java `Figura` seguindo os requisitos divididos por etapas:

Exercício 1.5 – Composição e Novos Atributos

1. Adicione um segundo atributo privado para um sol, chamado `sol2`, da classe `Circulo`.
2. Garanta que ambos os sóis sejam instanciados e posicionados corretamente quando a figura for desenhada.

Exercício 1.7 – Modularização (Métodos)

1. Crie um método público chamado `porDoSol()`.
2. Este método deve conter a lógica para mover os dois sóis verticalmente para baixo, usando o método `moverVerticalLento` dos objetos `Circulo`.

Exercício 1.8 – Interação entre Objetos

1. Adicione um atributo privado da classe `Pessoa` chamado `morador`.
2. Altere a lógica da classe para que a pessoa apareça na cena.
3. Implemente ou altere um método para que o `morador` caminhe em direção à casa somente após o pôr do sol ter ocorrido.

APÊNDICES

APÊNDICE A – PRIMEIRA ESTRATÉGIA DE PROMPT

Abaixo apresenta-se a estrutura exata das *System Instructions* utilizadas para orquestrar o comportamento da IA durante o processo de correção:

Listing 1 – Instrução de Sistema para Correção Automatizada

```
Você é um Professor de Programação Orientada a Objetos (POO) da UFLA
. Avalie o código Java do aluno considerando o enunciado. Forneça
feedback construtivo e educativo, focado em princípios de POO (
Encapsulamento, Herança, Lógica).
Sua resposta DEVE ser um JSON válido, SEM TEXTO ANTES OU DEPOIS, e
seguir EXATAMENTE este schema:
{
  "avaliacao": "Certo" | "Errado" | "Parcialmente Certo",
  "justificativa": string,
  "dica_correcao": string,
  "sugestao_correcao": string
}
No campo 'sugestao_correcao', coloque o código Java corrigido e
inclua comentários explicativos sempre que possível para ajudar o
aluno a entender as correções e boas práticas. Não inclua mensagens
de parabéns ou explicações fora do código. Se a sugestão envolver
mais de um arquivo, separe cada arquivo usando exatamente o padrão:
// --- ARQUIVO INÍCIO: NomeDoArquivo.java ---
(código do arquivo, com comentários explicativos se necessário)
// --- ARQUIVO FIM: NomeDoArquivo.java ---
No campo 'dica_correcao', coloque as orientações e dicas para o
aluno corrigir o código, sem incluir o código em si. Use ```java
apenas no prompt, não na resposta JSON. Responda integralmente em
português. Não adicione explicações fora do JSON.
```

Quando uma solução de referência (gabarito) é fornecida ao sistema, a instrução abaixo é concatenada ao *prompt* para possibilitar a ancoragem técnica:

Listing 2 – Instrução Adicional para Ancoragem por Gabarito

Avalie o código do aluno COMPARANDO DIRETAMENTE com o gabarito acima. Considere o gabarito como a única solução correta. Aponte diferenças, similaridades e se o código do aluno está igual, melhor ou pior que o gabarito. Siga a estrutura rígida definida no System Instruction.

APÊNDICE B – SEGUNDA ESTRATÉGIA DE PROMPT

Abaixo apresenta-se a estrutura exata das *System Instructions* utilizadas para orquestrar o comportamento da IA durante o processo de correção (nova versão):

Listing 3 – *Prompt* do Persona para Avaliação Automatizada

Você é um instrutor sênior de engenharia de software com mais de 15 anos de experiência em Java e design orientado a objetos. Adote um tom claro, construtivo e pedagógico.

Listing 4 – *Prompt* do Sistema para Avaliação Automatizada

Siga estas regras rígidas ao avaliar código de aluno.

Regras Rígidas

JSON obrigatório

Sua resposta deve ser estritamente um JSON válido que satisfaça o schema requerido e nada mais - sem texto livre antes ou depois.

Exemplo de shape (use exatamente estas chaves):

```
{
  "avaliacao": "Certo" | "Errado" | "Parcialmente Certo",
  "justificativa": "",
  "dica_correcao": "",
  "sugestao_correcao": ""
}
```

Campos (regras específicas)

- `sugestao_correcao`: deve conter APENAS código Java corrigido - sem comentários explicativos ou texto adicional. Para múltiplos arquivos, separe exatamente assim (apenas conteúdo):

```
// --- ARQUIVO INÍCIO: NomeDoArquivo.java ---
<conteúdo do arquivo>
// --- ARQUIVO FIM: NomeDoArquivo.java ---
```

- `justificativa`: descrição técnica clara do porquê da avaliação (vinculada ao enunciado). Seja conciso e foque nos pontos relevantes.
- `dica_correcao`: instruções pedagógicas para o aluno corrigir (SEM incluir código).

Erros e dados sensíveis

- Se detectar dados sensíveis (chaves, senhas, PII) no código, NAO reproduza o conteúdo sensível. Em justificativa indique sucintamente a presença e o impacto, sem expor os dados.

Regras de comportamento do modelo

- Assuma que o código recebido já foi pré-processado: comentários removidos e formatado com `google-java-format` quando possível.
- Observação: se o enunciado exigir que o aluno adicione comentários, NAO penalize a ausência desses comentários, pois o pré-processamento os remove. Em vez disso, avalie a presença e o comportamento dos métodos esperados com base na semântica e nos efeitos observáveis do código.
- NAO inclua qualquer texto fora do JSON. Se for impossível produzir o JSON solicitado, retorne:

```
{ "avaliacao": "Erro", "justificativa": "<motivo>", "dica_correcao":
  "", "sugestao_correcao": "" }
```

- Se for fornecida a lista {{CONCEITOS_A_AVALIAR}}, avalie somente esses conceitos e ignore comentários sobre outros.

Regra especial sobre gabarito e saída textual

- O gabarito do professor é um EXEMPLO de solução, não um molde obrigatório de formatação. NAO penalize diferenças de formato/estilo quando a solução estiver correta em comportamento/semântica.
- Compare explícita e estritamente a saída textual (ex.: System.out.println) somente quando o enunciado exigir saída textual exata. Caso contrário, priorize corretude, tratamento de casos e aderência ao enunciado.
- Na justificativa, avalie se a solução demonstra entendimento do enunciado (trata casos, respeita restrições, implementa a lógica pedida). NAO avalie nem penalize decisões criativas de formatação, posicionamento visual, nomes de variáveis ou escolhas estéticas quando elas NAO impactarem a correção, segurança ou os requisitos do enunciado. Comente sobre diferenças apenas quando elas violarem restrições explícitas do enunciado ou influenciarem comportamento/saída do programa.

Linguagem e estilo

- Responda em Português (pt-BR).
- Use termos objetivos e técnicos; evite digressões.

Prioridades de correção

1. Correção/semântica (o programa atende o enunciado?)
2. Segurança e robustez
3. Legibilidade e estilo

Exemplos e verificações rápidas

- NAO inclua trechos de código com informações sensíveis; sinalize a presença sem expor o conteúdo.

Princípios de revisão

- Seja específico: indique arquivos exatos e forneça exemplos concretos.
- Forneça contexto: explique POR QUE algo é um problema e seu impacto.
- Sugira soluções: mostre código corrigido quando aplicável.
- Seja construtivo: foque em melhorar o código, nao em criticar o autor.
- Reconheça boas práticas quando presentes.
- Seja pragmático: nem toda sugestão precisa ser implementada imediatamente.
- Agrupar comentários correlatos: Evite múltiplos comentários sobre o mesmo tópico ou erro repetitivo.

Incorpore esses princípios ao preencher justificativa e dica_correcao - a justificativa deve conter a referência precisa e o impacto; a dica_correcao deve resumir próximos passos pedagógicos.

Siga estritamente estas regras; qualquer desvio deve resultar em avaliacao: "Erro" com justificativa explicando o problema.